Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 1_MCQ

Attempt : 2
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. An algorithm must have a clear stopping point to ensure

Answer

Finiteness

Status : Correct ***Marks : 1/1***

2. Which of the following is true about algorithmic determinism?

Answer

Algorithms must produce the same output for the same input

Status : Correct ***Marks : 1/1***

vtu30672

3. An algorithm's effectiveness refers to

Answer

Each step contributes to solving the problem

Status : Correct ***Marks : 1/1***

4. Which notation is used to describe both the upper and lower bounds of an algorithm's efficiency?

Answer

Θ (Big theta)

Status : Correct ***Marks : 1/1***

5. What is the purpose of using asymptotic notations in the analysis of algorithm efficiency?

Answer

To take meaningful statements about an algorithm's efficiency based on its growth rate

Status : Correct ***Marks : 1/1***

6. What does the Big Theta notation (Θ) indicate about an algorithm's efficiency?

Answer

It describes the tightest possible bound on the algorithm's running time.

Status : Correct ***Marks : 1/1***

7. Which of the following statements is true regarding the relationship between Big Oh (O), Big Omega (Ω), and Big Theta (Θ) notations?

Answer

vtu30672

If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then $f(n) = \Theta(g(n))$

Status : Correct ***Marks : 1/1***

8. What does "Big O" notation represent in relation to algorithms?

Answer

A method to analyze and express the efficiency of an algorithm

Status : Correct ***Marks : 1/1***

9. An algorithm is required to produce a definite and unambiguous outcome for every valid input. This characteristic is known as _____.

Answer

determinism

Status : Correct ***Marks : 1/1***

10. Which approach is used to prove the correctness of algorithms by analyzing a sequence of steps?

Answer

Mathematical induction

Status : Correct ***Marks : 1/1***

11. In the context of algorithm design, what does the term "in-place" refer to?

Answer

The algorithm modifies the input data directly without using additional memory

Status : Correct ***Marks : 1/1***

12. What is the primary purpose of analyzing an algorithm's efficiency?

vtu30672

Answer

To optimize the algorithm's time and space efficiency

Status : Correct ***Marks : 1/1***

13. In the context of algorithmic problem-solving, what does the "generality of an algorithm" refer to?

Answer

The ability of an algorithm to handle a wide range of problem types

Status : Correct ***Marks : 1/1***

14. Which parameter is commonly used to measure an algorithm's input size?

Answer

Number of bits/characters representing the input

Status : Correct ***Marks : 1/1***

15. The concept of "Orders of Growth" in algorithm analysis refers to:

Answer

The function that describes the algorithm's efficiency

Status : Correct ***Marks : 1/1***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 2_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What is a non-recursive algorithm?

Answer

An algorithm that does not use recursion

Status : Correct ***Marks : 1/1***

2. Which of the following is an example of a non-recursive sorting algorithm?

Answer

Bubble Sort

Status : Correct ***Marks : 1/1***

vtu30672

3. The time complexity of non-recursive algorithms is often expressed using:

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

4. Which non-recursive algorithm is commonly used for searching in a sorted array by repeatedly dividing the search interval in half?

Answer

Binary Search

Status : Correct ***Marks : 1/1***

5. In non-recursive algorithms, iteration is commonly achieved through:

Answer

Loops

Status : Correct ***Marks : 1/1***

6. What is the output of the code?

```
#include <stdio.h>

int main() {
    int num = 2;
    do {
        printf("%d ", num);
        num *= 2;
    } while (num <= 8);
    return 0;
}
```

Answer

vtu30672

2 4 8

Status : Correct ***Marks : 1/1***

7. What is the output for the code?

```
#include <stdio.h>

void nonRecursiveAlgorithm(int n) {
    for (int i = 1; i <= n; i *= 2) {
        printf("%d ", i);
    }
}

int main() {
    nonRecursiveAlgorithm(8);
    return 0;
}
```

Answer

1 2 4 8

Status : Correct ***Marks : 1/1***

8. What is the output for the code?

```
#include <stdio.h>

void nonRecursiveFunction1(int n) {
    while (n > 0) {
        printf("%d ", n);
        n /= 2;
    }
}

int main() {
    int number = 80;
    nonRecursiveFunction1(number);
    return 0;
```

```
}
```

Answer

80 40 20 10 5 2 1

Status : Correct ***Marks : 1/1***

9. What will be the output for the following recursive code?

```
#include <stdio.h>

int power(int base, int exponent) {
    if (exponent == 0)
        return 1;
    else
        return base * power(base, exponent - 1);
}

int main() {
    int result = power(3, 2);
    printf("%d", result);
    return 0;
}
```

Answer

9

Status : Correct ***Marks : 1/1***

10. What will be the output for the following recursive code?

```
#include <stdio.h>

int num(int n, int rev) {
    if (n == 0) {
        return rev;
    } else {
        int digit = n % 10;
        rev = rev * 10 + digit;
```

```
    return num(n / 10, rev);
  }
}

int main() {
  int d = 12345;
  int rev = num(d, 0);
  printf("%d", rev);
  return 0;
}
```

Answer

54321

Status : Correct ***Marks : 1/1***

11. What will be the output for the following recursive code?

```
#include <stdio.h>

int function(int number) {
  if (number <= 0) {
    return 1;
  } else {
    return number * function(number - 1);
  }
}

int main() {
  int result;
  result = function(4);
  printf("%d", result);

  return 0;
}
```

Answer

24

Status : Correct ***Marks : 1/1***

12. What will be the output for the following recursive code?

```
#include <iostream>
using namespace std;

void print(int n) {
    if (n == 0)
        return;
    print(n / 2);
    cout << n % 2;
}

int main() {
    int num = 12;
    print(num);
    return 0;
}
```

Answer

1100

Status : Correct ***Marks : 1/1***

13. What will be the output for the following recursive code?

```
#include <stdio.h>

int function(int a, int b) {
    if (b == 1)
        return a;
    else
        return a + function(a, b - 1);
}

int main() {
    int result = function(2, 3);
    printf("%d", result);
    return 0;
}
```

Answer

6

Status : Correct *Marks : 1/1*

14. What will be the output for the following recursive code?

```
#include <stdio.h>

int function(int a, int b) {
    if (b == 0)
        return 0;
    if (b == 1)
        return a;
    return a + function(a, b - 1);
}

int main() {
    int result = function(3, 8);
    printf("%d", result);
    return 0;
}
```

Answer

24

Status : Correct *Marks : 1/1*

15. What will be the output for the following recursive code?

```
#include <iostream>
using namespace std;

int recursive(int n) {
    if (n <= 0)
        return 1;
    else
        return n + recursive(n - 1);
```

```
}

int main() {
    int result = recursive(5);
    cout << result;
    return 0;
}
```

Answer

16

Status : Correct ***Marks : 1/1***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 2_COD

Attempt : 2
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an integer n. The task is to find the first n Fibonacci Numbers.

Input Format

The input consists of a single integer n, representing how many Fibonacci terms to print.

Output Format

The output prints n space-separated Fibonacci numbers starting from 0.

Refer to the sample output for the formatting specifications.

vtu30672

Sample Test Case

Input: 3
Output: 0 1 1

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
   int n, a = 0, b = 1, next;

   scanf("%d", &n);

   for(int i = 0; i < n; i++) {
      printf("%d ", a);
      next = a + b;
      a = b;
      b = next;
   }

   return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given a string s, check if it is a palindrome using recursion. A palindrome is a word, phrase, or sequence that reads the same backward as forward.

Input Format

The input consists of a single string s containing no spaces.

Output Format

The output prints "The string is a palindrome" if the string is symmetric.

Otherwise, prints "The string is not a palindrome".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abba

Output: The string is a palindrome

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int isPalindrome(char str[], int start, int end) {
    if (start >= end)
        return 1;

    if (str[start] != str[end])
        return 0;

    return isPalindrome(str, start + 1, end - 1);
}

int main() {
    char str[51];

    scanf("%s", str);

    int len = strlen(str);

    if (isPalindrome(str, 0, len - 1))
        printf("The string is a palindrome");
    else
        printf("The string is not a palindrome");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Shilpa is learning about the Fibonacci sequence for her computer science class. She wants to generate the first N terms of the Fibonacci series.

Can you help Shilpa by writing a program that takes a number N and prints the first N terms of the Fibonacci series using recursion?

Input Format

The input consists of an integer N, representing the number of terms in the Fibonacci sequence that need to be printed.

Output Format

The output prints the first N Fibonacci numbers, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 0 1 1 2 3 5

Answer

```
import java.util.Scanner;

// You are using GCC

int fib(int n) {
   if (n == 0)
      return 0;
   if (n == 1)
      return 1;
   return fib(n - 1) + fib(n - 2);
}

public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int N = sc.nextInt();
      for(int i = 0; i < N; i++) {
         System.out.print(fib(i));
         if(i != N - 1) System.out.print(" ");
```

```
        }
        System.out.println();
    }
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ashish is preparing a mathematics assignment where he needs to calculate the factorial of a number. Since his teacher asked him to use recursion for this task, he plans to complete the method int factorial to solve it.

Help him to implement the task.

Input Format

The input consists of an integer num representing the number for which the factorial is to be calculated.

Output Format

The output prints the factorial value of num.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
Output: 6

Answer

```
#include <stdio.h>

// You are using GCC

int factorial(int n) {
    if (n == 0)
        return 1;
```

```
    return n * factorial(n - 1);
}

int main() {
    int num;
    scanf("%d", &num);
    printf("%d\n", factorial(num));
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 2_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Manoj loves puzzles and recently encountered an interesting mathematical challenge. He needs to find how many ways he can express a number x as the sum of distinct positive integers raised to the power of n. For example, for a given number x, he wants to find all possible ways to represent it as a sum of distinct powers of integers.

Help him implement this task.

Example 1

Input:

10
2

vtu30672

Output:

1

Explanation:

x = 10

n = 2

10 = 12 + 32, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

x = 100

n = 2

100 = 102 OR 62 + 82 OR 12 + 32 + 42 + 52 + 72, hence total 3 possibilities.

Function Specifications: getAllWays(int, int, int)

Input Format

The first line of input consists of an integer x, representing the target sum.

The second line of input consists of an integer n, representing the power to which the integers are raised.

Output Format

The first line of output prints: The total number of ways to express x as the sum of distinct powers.

vtu30672

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100
2

Output: 3

Answer

```
#include <stdio.h>
#include <math.h>

int countWays(int x, int n, int num) {
    int val = pow(num, n);

    if (x == 0) {
        return 1;
    }
    if (val > x) {
        return 0;
    }

    return countWays(x - val, n, num + 1) + countWays(x, n, num + 1);
}

int main() {
    int x, n;

    if (scanf("%d %d", &x, &n) == 2) {
        printf("%d\n", countWays(x, n, 1));
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given an integer array of coins of size N representing different types of

currency and an integer sum M. The task is to find the number of ways to make the sum by using different combinations from the coins array, using recursion.

Function Specifications: int countWays(int [], int , int)

Input Format

The first line of input consists of the number of coins N.

The second line consists of N integers denoting the coin denominations.

The third line consists of the integer sum M to be produced with those coins.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
4
Output: 4

Answer

```
#include <stdio.h>
int countWays(int coins[], int n, int m) {
    if (m == 0) {
        return 1;
    }

    if (m < 0 || n <= 0) {
        return 0;
    }

    return countWays(coins, n, m - coins[n - 1]) + countWays(coins, n - 1, m);
}
```

```
int main() {
    int n, m;
        if (scanf("%d", &n) != 1) return 0;

    int coins[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }

    if (scanf("%d", &m) != 1) return 0;

    printf("%d\n", countWays(coins, n, m));

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Meredith is a cashier at a local convenience store. To improve the efficiency of making change for customers, Meredith wants to develop a small program that calculates the number of different ways she can give change using a given set of coin denominations. This will help her quickly figure out the number of ways to make change for a particular amount of money using the available coins.

Meredith has a list of coin denominations and a specific amount of money for which she needs to provide change. She wants to write a program to determine the number of possible ways to achieve the given amount using the available denominations. This will help her understand all possible combinations without having to manually calculate them each time.

Input Format

The first line contains an integer n, the number of different coin denominations.

The second line contains n space-separated integers representing the coin denominations.

The third line contains an integer m, the target amount of money for which Meredith needs to make change.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
4

Output: 4

Answer

```
#include <stdio.h>
int countWays(int n, int coins[], int m) {
    if (m == 0) {
        return 1;
    }
    if (m < 0 || n <= 0) {
        return 0;
    }
    return countWays(n, coins, m - coins[n - 1]) + countWays(n - 1, coins, m);
}

int main() {
    int n, m;

    if (scanf("%d", &n) != 1) {
        return 0;
    }

    int coins[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }
```

```
    if (scanf("%d", &m) != 1) {
        return 0;
    }

    printf("%d\n", countWays(n, coins, m));

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
#include <stdio.h>

int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        if (n & 1) {
            count++;
        }
        n = n >> 1;
    }
    return count;
}

int main() {
    int n;
    if (scanf("%d", &n) == 1) {
        printf("%d\n", countSetBits(n));
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Raju is a computer graphics designer. He works around matrices with ease. His friend challenged him to rotate an M x M matrix clockwise with an angle of 90 degrees.

However, Raju was given a constraint that he must not use any additional 2D matrix for doing the above-mentioned task. The rotation must be done in place, which means Raju has to modify the given input matrix directly.

Example: 1

Input:

[[1,2,3],[4,5,6],[7,8,9]]

Output:

[[7,4,1],[8,5,2],[9,6,3]]

Example: 2

Input:

matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

Output:

[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Input Format

The first line of the input is a single integer value for the size of the row and column, M.

The next M lines of the input are the matrix elements, each containing M integers representing the elements of the matrix.

Output Format

The output displays the rotated matrix, with each row printed on a new line.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 3
1 2 3
4 5 6
7 8 9

Output: 7 4 1
8 5 2
9 6 3

vtu30672

Answer

```
#include <stdio.h>

void rotateMatrix(int m, int matrix[m][m]) {
    for (int i = 0; i < m; i++) {
        for (int j = i + 1; j < m; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m / 2; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[i][m - 1 - j];
            matrix[i][m - 1 - j] = temp;
        }
    }
}

int main() {
    int m;
    if (scanf("%d", &m) != 1) return 0;

    int matrix[m][m];

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    rotateMatrix(m, matrix);

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 2_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

In a kingdom far away, two types of soup are prepared daily — Soup A and Soup B. The royal chef always starts with n milliliters of each soup. Each time a servant is served, one of the following four serving operations is chosen at random (each with equal probability of 0.25):

Serve 100 ml of Soup A and 0 ml of Soup B

Serve 75 ml of Soup A and 25 ml of Soup B

Serve 50 ml of Soup A and 50 ml of Soup B

Serve 25 ml of Soup A and 75 ml of Soup B

The serving continues until either or both soups become empty. If the required soup amount for an operation exceeds the available quantity,

serve as much as possible (up to what remains).

You are to compute the probability that Soup A becomes empty first, plus half the probability that both soups become empty at the same time.

Special Note:

Since all operations serve soup in multiples of 25 ml, you must first scale n down to units of 25 ml before recursive processing:

Example

Sample Input 1

n = 50

Sample Output 2

0.625

Explanation

If we choose the first two operations, A will become empty first.

For the third operation, A and B will become empty at the same time.

For the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is 0.25 * (1 + 1 + 0.5 + 0) = 0.625.

Input Format

The input consists of an integer n, representing the total volume of the soup.

Output Format

The output prints the probability that soup A will be empty first, plus half the probability that A and B become empty at the same time.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

Output: 0.625

Answer

```
#include <stdio.h>
#include <string.h>
double dp[101][101];
double solve(int a, int b) {

    if (a <= 0 && b <= 0) return 0.5;
    if (a <= 0) return 1.0;
    if (b <= 0) return 0.0;
    if (dp[a][b] >= 0) return dp[a][b];
    dp[a][b] = 0.25 * (
        solve(a - 4, b) +
        solve(a - 3, b - 1) +
        solve(a - 2, b - 2) +
        solve(a - 1, b - 3)
    );

    return dp[a][b];
}

int main() {
    int n;
    scanf("%d", &n);
    int units = (n + 24) / 25;
    for (int i = 0; i <= 100; i++) {
        for (int j = 0; j <= 100; j++) {
            dp[i][j] = -1.0;
        }
    }

    double result = solve(units, units);

    printf("%.3f", result);

    return 0;
}
```

Status : Correct *Marks : 10/10*

2. Problem Statement

In a unique team-building exercise, a company has decided to distribute a certain number of resources among its departments based on mathematical powers. The company has x resources that need to be distributed such that each department receives a different power of an integer. Specifically, each department i receives n resources, where n is a given power.

George, the resource manager, needs to find all possible ways to distribute exactly x resources using the powers of integers starting from 1. Each integer's power can only be used once.

Write a program to help George determine the number of ways to distribute x resources in this manner using a recursive manner.

Example 1

Input:

10

2

Output:

1

Explanation:

x = 10

n = 2

10 = 12 + 32, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

x = 100

n = 2

100 = 102 OR 62 + 82 OR 12 + 32 + 42 + 52 + 72, hence total 3 possibilities.

Input Format

The first line of input consists of the integer x, representing the total number of resources.

The second line consists of the integer n, representing the power to which the integers are raised.

Output Format

The output should be a single integer representing the number of ways to distribute exactly x resources.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100
2

Output: 3

Answer

```
#include <stdio.h>
#include <math.h>
int countWays(int x, int n, int num) {
    int power = pow(num, n);
    if (power == x) return 1;
    if (power > x) return 0;
    return countWays(x - power, n, num + 1) +
```

```
    countWays(x, n, num + 1);
}

int main() {
    int x, n;
    scanf("%d", &x);
    scanf("%d", &n);

    int result = countWays(x, n, 1);

    printf("%d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
#include <stdio.h>
int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        if (n & 1)
            count++;
        n = n >> 1;
    }
    return count;
}
int main() {
    int n;
    scanf("%d", &n);
    printf("%d", countSetBits(n));
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given a square matrix of size n×n, rotate the elements along each layer (or boundary) of the matrix by one position in a clockwise direction. The rotation should be performed layer by layer, starting from the outermost layer and proceeding inward.

Examples:

Input 2:

3 3

1 2 3

4 5 6

7 8 9

Output 2:

4 1 2

7 5 3

8 9 6

Explanation:

For a clockwise boundary rotation by one step:

We start with the outermost layer (there's only one layer in a 3×3 matrix)

We save the top-left element (1)

Move left edge upward: 4 1, 7 4

Move bottom edge leftward: 8 7, 9 8

Move right edge downward: 6 9, 3 6

Move top edge rightward: 2 3

Place the saved element (1) in the second position of the top row, so 1 2

Input Format

The first line contains two integers R and C — the number of rows and columns in the matrix.

The next R lines each contain C integers, representing the elements of the matrix.

Output Format

The output displays the rotated matrix after performing the clockwise boundary rotation.

Refer to the sample input and output for formatting specifications.

Sample Test Case

vtu30672

Input: 4 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

Output: 5 1 2 3
9 10 6 4
13 11 7 8
14 15 16 12

Answer

```
#include <stdio.h>
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    int a[5][5];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    int top = 0, bottom = n - 1;
    int left = 0, right = m - 1;
    while (top < bottom && left < right) {
        int prev = a[top + 1][left];
        for (int i = left; i <= right; i++) {
            int temp = a[top][i];
            a[top][i] = prev;
            prev = temp;
        }
        top++;
        for (int i = top; i <= bottom; i++) {
            int temp = a[i][right];
            a[i][right] = prev;
            prev = temp;
        }
        right--;
        for (int i = right; i >= left; i--) {
            int temp = a[bottom][i];
            a[bottom][i] = prev;
            prev = temp;
        }
```

```
        bottom--;
        for (int i = bottom; i >= top; i--) {
            int temp = a[i][left];
            a[i][left] = prev;
            prev = temp;
        }
        left++;
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Given a 2D matrix of size ROW × COL, print all elements of the matrix in diagonal order.

Diagonal Order Definition:

Start from the first element at position (0,0).Traverse all diagonals starting from the leftmost column going downward, then continue with diagonals starting from the bottom row going rightward.Within each diagonal, print elements from the top-right corner to bottom-left corner.

Example:

Input matrix

5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Diagonal printing of the above matrix is

1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

Input Format

The first line of input contains two integers, ROW and COL.

The next ROW lines each contain COL space-separated integers

Output Format

The output consists of multiple lines, one for each diagonal. Each line contains the matrix elements of that diagonal in the order from top-right to bottom-left, separated by spaces.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
Output: 1
5 2

vtu30672

9 6 3
13 10 7 4
17 14 11 8
18 15 12
19 16
20

Answer

```
#include <stdio.h>

int main() {
    int ROW, COL;
    scanf("%d %d", &ROW, &COL);

    int a[10][10];
    for (int i = 0; i < ROW; i++) {
        for (int j = 0; j < COL; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    for (int start = 0; start < ROW; start++) {
        int i = start;
        int j = 0;

        while (i >= 0 && j < COL) {
            printf("%d ", a[i][j]);
            i--;
            j++;
        }
        printf("\n");
    }
    for (int start = 1; start < COL; start++) {
        int i = ROW - 1;
        int j = start;

        while (i >= 0 && j < COL) {
            printf("%d ", a[i][j]);
            i--;
            j++;
        }
        printf("\n");
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 1_MCQ

Attempt : 2
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. An algorithm must have a clear stopping point to ensure

Answer

Finiteness

Status : Correct ***Marks : 1/1***

2. Which of the following is true about algorithmic determinism?

Answer

Algorithms must produce the same output for the same input

Status : Correct ***Marks : 1/1***

vtu30672

3. An algorithm's effectiveness refers to

Answer

Each step contributes to solving the problem

Status : Correct ***Marks : 1/1***

4. Which notation is used to describe both the upper and lower bounds of an algorithm's efficiency?

Answer

Θ (Big theta)

Status : Correct ***Marks : 1/1***

5. What is the purpose of using asymptotic notations in the analysis of algorithm efficiency?

Answer

To take meaningful statements about an algorithm's efficiency based on its growth rate

Status : Correct ***Marks : 1/1***

6. What does the Big Theta notation (Θ) indicate about an algorithm's efficiency?

Answer

It describes the tightest possible bound on the algorithm's running time.

Status : Correct ***Marks : 1/1***

7. Which of the following statements is true regarding the relationship between Big Oh (O), Big Omega (Ω), and Big Theta (Θ) notations?

Answer

vtu30672

If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then $f(n) = \Theta(g(n))$

Status : Correct ***Marks : 1/1***

8. What does "Big O" notation represent in relation to algorithms?

Answer

A method to analyze and express the efficiency of an algorithm

Status : Correct ***Marks : 1/1***

9. An algorithm is required to produce a definite and unambiguous outcome for every valid input. This characteristic is known as _____.

Answer

determinism

Status : Correct ***Marks : 1/1***

10. Which approach is used to prove the correctness of algorithms by analyzing a sequence of steps?

Answer

Mathematical induction

Status : Correct ***Marks : 1/1***

11. In the context of algorithm design, what does the term "in-place" refer to?

Answer

The algorithm modifies the input data directly without using additional memory

Status : Correct ***Marks : 1/1***

12. What is the primary purpose of analyzing an algorithm's efficiency?

vtu30672

Answer

To optimize the algorithm's time and space efficiency

Status : Correct ***Marks : 1/1***

13. In the context of algorithmic problem-solving, what does the "generality of an algorithm" refer to?

Answer

The ability of an algorithm to handle a wide range of problem types

Status : Correct ***Marks : 1/1***

14. Which parameter is commonly used to measure an algorithm's input size?

Answer

Number of bits/characters representing the input

Status : Correct ***Marks : 1/1***

15. The concept of "Orders of Growth" in algorithm analysis refers to:

Answer

The function that describes the algorithm's efficiency

Status : Correct ***Marks : 1/1***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 3_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 28

Section 1 : Coding

1. Problem Statement

Adhi is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Adhi is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Adhi by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

vtu30672

N = 7,

Points: {(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)},

Query: X = 3, Y = 2

Below is the image of the plotting of the given points:

The query (3, 2) lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N, representing the number of points in the set.

The following N lines, each containing two integers x and y, represent the coordinates of the points in the set.

The last line consists of two integers px and py, representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
1 1
2 1
3 1
4 1
4 2
4 3
4 4
3 2

vtu30672

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int x, y;
} Point;
int cross(Point O, Point A, Point B) {
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}
int compare(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;
    if (p1->x == p2->x)
        return p1->y - p2->y;
    return p1->x - p2->x;
}
int convexHull(Point points[], int n, Point hull[]) {
    qsort(points, n, sizeof(Point), compare);

    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(hull[k-2], hull[k-1], points[i]) <= 0)
            k--;
        hull[k++] = points[i];
    }
    int t = k + 1;
    for (int i = n - 2; i >= 0; i--) {
        while (k >= t && cross(hull[k-2], hull[k-1], points[i]) <= 0)
            k--;
        hull[k++] = points[i];
    }

    return k - 1;
}
int isInside(Point hull[], int m, Point p) {
    for (int i = 0; i < m; i++) {
        Point A = hull[i];
        Point B = hull[(i + 1) % m];
```

```
        if (cross(A, B, p) < 0)
            return 0;
    }
    return 1;
}

int main() {
    int n;
    scanf("%d", &n);

    Point points[20], hull[20];

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    Point p;
    scanf("%d %d", &p.x, &p.y);

    int m = convexHull(points, n, hull);

    if (isInside(hull, m, p))
        printf("The point (%d, %d) lies inside the Convex Hull.", p.x, p.y);
    else
        printf("The point (%d, %d) lies outside the Convex Hull.", p.x, p.y);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Olivia, a city planner, is designing a recreational area surrounded by specific landmarks. To ensure balance in the design, Olivia needs to calculate the weighted centroid of the boundary formed by these landmarks using the convex hull.

Your task is to compute the convex hull and determine the weighted centroid of its vertices.

Note:

MAXN (100000): The maximum capacity for both the points and hull arrays, allowing storage of up to 100,000 points each.1e-9 is the threshold for floating-point comparisons.

Example:

Input:

6

0 0

4 0

4 3

0 3

2 1

3 2

Output:

Weighted Centroid: 2.00 1.50

Explanation:

Input Points: (0, 0), (2, 0), (2, 2), (0, 2), (1, 1), (3, 1).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two floating-point numbers x and y, representing the coordinates of the points.

vtu30672

Output Format

The output prints the coordinates of the weighted centroid of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
0 0
2 0
2 2
0 2
1 1
Output: Weighted Centroid: 1.00 1.00

Answer

```
#include <stdio.h>
#include <stdlib.h>
#define MAXN 100000
typedef struct {
   double x, y;
} Point;
Point points[MAXN], hull[MAXN];
double cross(Point O, Point A, Point B) {
   return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}
int cmp(const void *a, const void *b) {
   Point *p1 = (Point *)a;
   Point *p2 = (Point *)b;
   if (p1->x == p2->x)
      return (p1->y > p2->y) - (p1->y < p2->y);
   return (p1->x > p2->x) - (p1->x < p2->x);
}
int convexHull(int n) {
   qsort(points, n, sizeof(Point), cmp);
   int k = 0;
   for (int i = 0; i < n; i++) {
      while (k >= 2 && cross(hull[k-2], hull[k-1], points[i]) <= 0)
```

```
        k--;
        hull[k++] = points[i];
    }
    int t = k + 1;
    for (int i = n - 2; i >= 0; i--) {
        while (k >= t && cross(hull[k-2], hull[k-1], points[i]) <= 0)
            k--;
        hull[k++] = points[i];
    }
    return k - 1;
}
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }
    int hullSize = convexHull(n);
    double sumX = 0, sumY = 0;
    for (int i = 0; i < hullSize; i++) {
        sumX += hull[i].x;
        sumY += hull[i].y;
    }
    double cx = sumX / hullSize;
    double cy = sumY / hullSize;
    printf("Weighted Centroid: %.2lf %.2lf", cx, cy);
    return 0;
}
```

Status : Partially correct **Marks : 6/10**

3. Problem Statement:

Gwen, a city planner, is designing a new park and needs to calculate the perimeter of the park's boundary. She has the coordinates of key boundary points and wants to find the minimum perimeter that encloses all these points.

Using the convex hull, Gwen needs help in computing both the boundary and its perimeter.

Note:

points[100000]: An array that can store up to 100,000 input points.hull[100000]: An array that can store up to 100,000 points on the convex hull.1e-9: A threshold for floating-point comparisons to handle precision issues.

Example

Input:

6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output:

Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Explanation:

Input Points: (0.0, 0.0), (3.0, 0.0), (3.0, 2.0), (1.0, 2.0), (0.0, 2.0), (2.0, 1.0).

Convex Hull Construction:

Pivot: Start with the lowest point, (0, 0).Sorted by Polar Angle: The points are sorted, and counterclockwise turns are included in the hull.Convex Hull Points: (0, 0), (3, 0), (3, 2), (0, 2).Perimeter Calculation:

vtu30672

Distances between consecutive hull points:(0, 0)-(3, 0): 3.00, (3, 0)-(3, 2): 2.00, (3, 2)-(0, 2): 3.00, (0, 2)-(0, 0): 2.00.Total Perimeter: 10.00.

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two space separated decimal values x and y, representing the coordinates of the points.

Output Format

The first line prints "Convex Hull:" followed by the coordinates of the points on the convex hull, each on a new line, in counterclockwise order.

The last line prints the "Perimeter: ", followed by the total perimeter of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
0.0 0.0
3.0 0.0
3.0 2.0
1.0 2.0
0.0 2.0
2.0 1.0
Output: Convex Hull:
0 0
3 0
3 2
0 2
Perimeter: 10.00

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```
typedef struct {
    double x, y;
} Point;

Point points[100], hull[100];
double cross(Point O, Point A, Point B) {
    return (A.x - O.x)*(B.y - O.y) - (A.y - O.y)*(B.x - O.x);
}
int cmp(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;

    if (p1->x == p2->x)
        return (p1->y > p2->y) - (p1->y < p2->y);
    return (p1->x > p2->x) - (p1->x < p2->x);
}
double dist(Point A, Point B) {
    return sqrt((A.x - B.x)*(A.x - B.x) + (A.y - B.y)*(A.y - B.y));
}
int convexHull(int n) {
    qsort(points, n, sizeof(Point), cmp);

    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(hull[k-2], hull[k-1], points[i]) <= 0)
            k--;
        hull[k++] = points[i];
    }
    int t = k + 1;
    for (int i = n - 2; i >= 0; i--) {
        while (k >= t && cross(hull[k-2], hull[k-1], points[i]) <= 0)
            k--;
        hull[k++] = points[i];
    }

    return k - 1;
}

int main() {
    int n;
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }

    int m = convexHull(n);

    printf("Convex Hull:\n");

    for (int i = 0; i < m; i++) {
        printf("%.0lf %.0lf\n", hull[i].x, hull[i].y);
    }
    double perimeter = 0;
    for (int i = 0; i < m; i++) {
        perimeter += dist(hull[i], hull[(i + 1) % m]);
    }

    printf("Perimeter: %.2lf", perimeter);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement:

Maria, a wildlife researcher, is studying the distribution of endangered species in a forest. She has mapped several points that represent observation locations inside the forest.

Maria wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and she needs to rank them by their distance to the outer boundary of the forest.

Help Maria compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull

arrays.1e-9:Threshold for floating-point equality checks to handle precision errors.1e-12: Threshold to detect degenerate (zero-length) segments.1e30:Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3
0 0
4 0
4 4
0 4
2 2
1 2
3 2

Output: 2 2
3 2
1 2

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
   double x, y;
} Point;

typedef struct {
   Point p;
   double dist;
} PointDist;

Point p0;
```

```
double distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2;
}

int comparePoints(const void* vp1, const void* vp2) {
    Point* p1 = (Point*)vp1;
    Point* p2 = (Point*)vp2;
    int o = orientation(p0, *p1, *p2);
    if (o == 0)
        return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}
int compareDist(const void* a, const void* b) {
    PointDist* pd1 = (PointDist*)a;
    PointDist* pd2 = (PointDist*)b;

    if (fabs(pd1->dist - pd2->dist) > 1e-9)
        return (pd1->dist > pd2->dist) ? -1 : 1;
    if (fabs(pd1->p.x - pd2->p.x) > 1e-9)
        return (pd1->p.x > pd2->p.x) ? -1 : 1;

    return (pd1->p.y > pd2->p.y) ? -1 : 1;
}

double getDistToSegment(Point p, Point a, Point b) {
    double l2 = distSq(a, b);
    if (l2 < 1e-12) return sqrt(distSq(p, a));
    double t = ((p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y)) / l2;
    if (t < 0) t = 0;
    else if (t > 1) t = 1;
    Point proj = {a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};
    return sqrt(distSq(p, proj));
}

int main() {
    int N, K;
```

```
    if (scanf("%d %d", &N, &K) != 2) return 0;

    Point pts[N], original[N];
    for (int i = 0; i < N; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
        original[i] = pts[i];
    }
    int ymin = 0;
    for (int i = 1; i < N; i++) {
        if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))
            ymin = i;
    }
    Point temp = pts[0]; pts[0] = pts[ymin]; pts[ymin] = temp;
    p0 = pts[0];
    qsort(&pts[1], N - 1, sizeof(Point), comparePoints);

    Point hull[N];
    int hSize = 0;
    for (int i = 0; i < N; i++) {
        while (hSize >= 2 && orientation(hull[hSize - 2], hull[hSize - 1], pts[i]) != 2)
            hSize--;
        hull[hSize++] = pts[i];
    }

    PointDist res[N];
    for (int i = 0; i < N; i++) {
        double minDist = 1e30;
        for (int j = 0; j < hSize; j++) {
            double d = getDistToSegment(original[i], hull[j], hull[(j + 1) % hSize]);
            if (d < minDist) minDist = d;
        }
        res[i].p = original[i];
        res[i].dist = minDist;
    }

    qsort(res, N, sizeof(PointDist), compareDist);

    for (int i = 0; i < K; i++) {
        printf("%g %g\n", res[i].p.x, res[i].p.y);
    }

    return 0;
```

}

Status : Partially correct ***Marks : 1/10***

5. Problem Statement:

Henry, a wildlife researcher, is studying the distribution of endangered species in a forest. He has mapped several points that represent observation locations inside the forest.

Henry wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and he needs to rank them by their distance to the outer boundary of the forest.

Help Henry compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays.1e-9:Threshold for floating-point equality checks to handle precision errors.1e-12: Threshold to detect degenerate (zero-length) segments.1e30:Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3
0 0
4 0
4 4
0 4
2 2

vtu30672

1 2
3 2
Output: 2 2
3 2
1 2

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    double x, y;
} Point;

typedef struct {
    Point p;
    double dist;
} PointDist;

Point p0;

double distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2;
}

int comparePoints(const void* vp1, const void* vp2) {
    Point* p1 = (Point*)vp1;
    Point* p2 = (Point*)vp2;
    int o = orientation(p0, *p1, *p2);
    if (o == 0)
        return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}
int compareDist(const void* a, const void* b) {
    PointDist* pd1 = (PointDist*)a;
```

```
    PointDist* pd2 = (PointDist*)b;

    if (fabs(pd1->dist - pd2->dist) > 1e-9)
        return (pd1->dist > pd2->dist) ? -1 : 1;
    if (fabs(pd1->p.x - pd2->p.x) > 1e-9)
        return (pd1->p.x > pd2->p.x) ? -1 : 1;

    return (pd1->p.y > pd2->p.y) ? -1 : 1;
}

double getDistToSegment(Point p, Point a, Point b) {
    double l2 = distSq(a, b);
    if (l2 < 1e-12) return sqrt(distSq(p, a));
    double t = ((p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y)) / l2;
    if (t < 0) t = 0;
    else if (t > 1) t = 1;
    Point proj = {a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};
    return sqrt(distSq(p, proj));
}

int main() {
    int N, K;
    if (scanf("%d %d", &N, &K) != 2) return 0;

    Point pts[N], original[N];
    for (int i = 0; i < N; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
        original[i] = pts[i];
    }
    int ymin = 0;
    for (int i = 1; i < N; i++) {
        if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))
            ymin = i;
    }
    Point temp = pts[0]; pts[0] = pts[ymin]; pts[ymin] = temp;
    p0 = pts[0];
    qsort(&pts[1], N - 1, sizeof(Point), comparePoints);

    Point hull[N];
    int hSize = 0;
    for (int i = 0; i < N; i++) {
        while (hSize >= 2 && orientation(hull[hSize - 2], hull[hSize - 1], pts[i]) != 2)
```

```
        hSize--;
    hull[hSize++] = pts[i];
    }
    PointDist res[N];
    for (int i = 0; i < N; i++) {
        double minDist = 1e30;
        for (int j = 0; j < hSize; j++) {
            double d = getDistToSegment(original[i], hull[j], hull[(j + 1) % hSize]);
            if (d < minDist) minDist = d;
        }
        res[i].p = original[i];
        res[i].dist = minDist;
    }

    qsort(res, N, sizeof(PointDist), compareDist);

    for (int i = 0; i < K; i++) {
        printf("%g %g\n", res[i].p.x, res[i].p.y);
    }

    return 0;
}
```

Status : Partially correct ***Marks : 1/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 3_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement:

Javier, a landscape architect, is designing a new botanical garden and needs to determine the outer boundary that will enclose all the selected plant species.

He has gathered coordinates of key points where the plants are located and needs to calculate the convex hull to create a protective garden boundary. To determine this boundary, Javier decides to use Graham's Scan algorithm.

Help Javier compute the convex hull by implementing the algorithm to design the garden's boundary.

Input Format

vtu30672

The first line of input consists of an integer n, representing the number of plant species locations.

The next n lines contain two integers x and y, representing the coordinates of each location.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the points forming the convex hull in counterclockwise order, with each point on a new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 0
0 1
1 0
1 1

Output: Convex Hull:
(0, 0)
(1, 0)
(1, 1)
(0, 1)

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MAXN 10
typedef struct {
    int x, y;
} Point;

Point points[MAXN], hull[MAXN];
int n;
Point pivot;
int cross(Point O, Point A, Point B) {
    return (A.x - O.x)*(B.y - O.y) - (A.y - O.y)*(B.x - O.x);
}
```

```
int dist2(Point A, Point B) {
    return (A.x - B.x)*(A.x - B.x) + (A.y - B.y)*(A.y - B.y);
}
int cmp(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;

    int c = cross(pivot, *p1, *p2);
    if (c == 0) {
        return dist2(pivot, *p1) - dist2(pivot, *p2);
    }
    return (c > 0) ? -1 : 1;
}
int grahamScan() {
    pivot = points[0];
    for (int i = 1; i < n; i++) {
        if (points[i].y < pivot.y || (points[i].y == pivot.y && points[i].x < pivot.x)) {
            pivot = points[i];
        }
    }
    qsort(points, n, sizeof(Point), cmp);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(hull[k-2], hull[k-1], points[i]) <= 0)
            k--;
        hull[k++] = points[i];
    }

    return k;
}
int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    int hullSize = grahamScan();

    printf("Convex Hull:\n");
    for (int i = 0; i < hullSize; i++) {
        printf("(%d, %d)\n", hull[i].x, hull[i].y);
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Eva, a botanist, is surveying a nature reserve to protect rare plant species. During her survey, she marked a special tree, known as the "guardian tree," and now needs to map out the perimeter of the reserve, ensuring the guardian tree is included in the boundary. To do this, Eva needs to compute the convex hull of all the points, including the coordinates of the guardian tree.

Your task is to compute the convex hull, ensuring that the special guardian tree is included in the boundary.

Example

Input:

5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output:

0 0

2 0

3 1

2 2
0 2

Explanation:

Input Points: (0.0, 0.0), (2.0, 0.0), (2.0, 2.0), (0.0, 2.0), (1.0, 1.0), (3.0, 1.0).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of trees originally surveyed.

The next N lines each contain two double values x and y, representing the coordinates of the surveyed trees.

The last line contains two double values x and y, representing the coordinates of the guardian tree.

Output Format

The output prints the coordinates of the points forming the convex hull in counterclockwise order, ensuring the special guardian tree is included. Each coordinate should be printed on a new line, with the values of x and y separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
0.0 0.0
2.0 0.0
2.0 2.0

0.0 2.0
1.0 1.0
3.0 1.0

Output: 0 0
2 0
3 1
2 2
0 2

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MAXN 25
typedef struct {
   double x, y;
} Point;

Point points[MAXN], hull[MAXN];
int n;
Point pivot;
double cross(Point O, Point A, Point B) {
   return (A.x - O.x)*(B.y - O.y) - (A.y - O.y)*(B.x - O.x);
}
double dist2(Point A, Point B) {
   return (A.x - B.x)*(A.x - B.x) + (A.y - B.y)*(A.y - B.y);
}
int cmp(const void *a, const void *b) {
   Point *p1 = (Point*)a;
   Point *p2 = (Point*)b;
   double c = cross(pivot, *p1, *p2);
   if (fabs(c) < 1e-9) {
      double d1 = dist2(pivot, *p1);
      double d2 = dist2(pivot, *p2);
      return (d1 < d2) ? -1 : 1;
   }
   return (c > 0) ? -1 : 1;
}
int grahamScan() {
   pivot = points[0];
   for (int i = 1; i < n; i++) {
      if (points[i].y < pivot.y || (fabs(points[i].y - pivot.y) < 1e-9 && points[i].x <
pivot.x))
```

```
        pivot = points[i];
    }
    qsort(points, n, sizeof(Point), cmp);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(hull[k-2], hull[k-1], points[i]) <= 1e-9)
            k--;
        hull[k++] = points[i];
    }
    return k;
}

int main() {
    int i;
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }
    Point guardian;
    scanf("%lf %lf", &guardian.x, &guardian.y);
    points[n++] = guardian;

    int hullSize = grahamScan();

    for (i = 0; i < hullSize; i++) {
        printf("%.0f %.0f\n", hull[i].x, hull[i].y);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Nathan is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Nathan is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Nathan by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

N = 7,

Points: {(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)},

Query: X = 3, Y = 2

Below is the image of the plotting of the given points:

The query (3, 2) lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N, representing the number of points in the set.

The following N lines, each containing two integers x and y, represent the coordinates of the points in the set.

The last line consists of two integers px and py, representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

vtu30672

1 1
2 1
3 1
4 1
4 2
4 3
4 4
3 2

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MAXN 15
typedef struct {
    int x, y;
} Point;

Point points[MAXN], hull[MAXN];
int n;
Point pivot;
int cross(Point O, Point A, Point B) {
    return (A.x - O.x)*(B.y - O.y) - (A.y - O.y)*(B.x - O.x);
}
int dist2(Point A, Point B) {
    return (A.x - B.x)*(A.x - B.x) + (A.y - B.y)*(A.y - B.y);
}
int cmp(const void *a, const void *b) {
    Point *p1 = (Point*)a;
    Point *p2 = (Point*)b;
    int c = cross(pivot, *p1, *p2);
    if (c == 0) {
        int d1 = dist2(pivot, *p1);
        int d2 = dist2(pivot, *p2);
        return (d1 < d2) ? -1 : 1;
    }
    return (c > 0) ? -1 : 1;
}
int grahamScan() {
    pivot = points[0];
    for (int i = 1; i < n; i++) {
```

```
        if (points[i].y < pivot.y || (points[i].y == pivot.y && points[i].x < pivot.x))
            pivot = points[i];
    }
    qsort(points, n, sizeof(Point), cmp);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(hull[k-2], hull[k-1], points[i]) <= 0)
            k--;
        hull[k++] = points[i];
    }
    return k;
}
int isInside(Point P, int m) {
    for (int i = 0; i < m; i++) {
        Point A = hull[i];
        Point B = hull[(i+1)%m];
        if (cross(A, B, P) < 0)
            return 0;
    }
    return 1;
}

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d %d", &points[i].x, &points[i].y);

    Point query;
    scanf("%d %d", &query.x, &query.y);

    int hullSize = grahamScan();

    if (isInside(query, hullSize))
        printf("The point (%d, %d) lies inside the Convex Hull.\n", query.x, query.y);
    else
        printf("The point (%d, %d) lies outside the Convex Hull.\n", query.x, query.y);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N, representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y, represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 3
1 1
10 18
4 9

Output: Closest pair of points: (0, 3) and (1, 1)
Distance: 2.24

Answer

vtu30672

```
#include <stdio.h>
#include <math.h>
typedef struct {
    double x, y;
} Point;
int main() {
    int n;
    scanf("%d", &n);
    Point points[n];
        for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }
    double minDist = 1e9;
    int idx1 = 0, idx2 = 1;
    for (int i = 0; i < n; i++) {
        for (int j = i+1; j < n; j++) {
            double dx = points[i].x - points[j].x;
            double dy = points[i].y - points[j].y;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < minDist) {
                minDist = dist;
                idx1 = i;
                idx2 = j;
            }
        }
    }
        printf("Closest pair of points: (%.0lf, %.0lf) and (%.0lf, %.0lf)\n",
            points[idx1].x, points[idx1].y,
            points[idx2].x, points[idx2].y);
             printf("Distance: %.2lf\n", minDist);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement:

Lena, a geologist, is studying the boundaries of a newly discovered forest reserve. To define the outer edges of the reserve, Lena needs to identify the convex hull from a set of points marking different trees within the area.

Using Graham's Scan algorithm, help Lena compute the convex hull to outline the forest reserve's perimeter.

Example1

Input:

6

0 0

1 1

2 2

4 4

0 4

4 0

Output:

Convex Hull:

(0, 0)

(4, 0)

(4, 4)

(0, 4)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (4, 0), (4, 4), (0, 4), (1, 1), (2, 2).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Example2

vtu30672

Input:

3

0 0

0 3

3 0

Output:

Convex Hull:

(0, 0)

(3, 0)

(0, 3)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (3, 0), (0, 3).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (3, 0), (0, 3).

Input Format

The first line of input contains an integer n, representing the number of trees in the forest reserve.

The next n lines contain two integers x and y, representing the coordinates of each tree.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the trees forming the convex hull in counterclockwise order in new line.

Refer to the sample output for formatting specifications.

vtu30672

Sample Test Case

Input: 6
0 0
1 1
2 2
4 4
0 4
4 0

Output: Convex Hull:
(0, 0)
(4, 0)
(4, 4)
(0, 4)

Answer

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    int x, y;
} Point;

Point pivot;
int orientation(Point p, Point q, Point r) {
    int val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
    if (val == 0) return 0;
    return (val > 0)? 1: 2;
}
void swap(Point *a, Point *b) {
    Point temp = *a;
    *a = *b;
    *b = temp;
}
int distSq(Point p1, Point p2) {
    int dx = p1.x - p2.x;
    int dy = p1.y - p2.y;
    return dx*dx + dy*dy;
}
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;
```

```
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}
void convexHull(Point points[], int n) {
    int ymin = points[0].y, min = 0;
    for (int i = 1; i < n; i++) {
        int y = points[i].y;
        if ((y < ymin) || (y == ymin && points[i].x < points[min].x)) {
            ymin = points[i].y;
            min = i;
        }
    }
    swap(&points[0], &points[min]);
    pivot = points[0];
    qsort(&points[1], n-1, sizeof(Point), compare);
    Point hull[n];
    int m = 0;
    hull[m++] = points[0];
    hull[m++] = points[1];
    hull[m++] = points[2];
    for (int i = 3; i < n; i++) {
        while (m >= 2 && orientation(hull[m-2], hull[m-1], points[i]) != 2)
            m--;
        hull[m++] = points[i];
    }
    printf("Convex Hull:\n");
    for (int i = 0; i < m; i++) {
        printf("(%d, %d)\n", hull[i].x, hull[i].y);
    }
}
int main() {
    int n;
    scanf("%d", &n);
    Point points[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }
    if (n < 3) {
        printf("Convex Hull:\n");
        for (int i = 0; i < n; i++)
```

```
        printf("(%d, %d)\n", points[i].x, points[i].y);
    return 0;
  }
  convexHull(points, n);
  return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 4_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. Which of the following methods can be used to solve the Knapsack problem?

Answer

All the mentioned options

Status : Correct ***Marks : 1/1***

2. 0/1 knapsack is based on _____method.

Answer

Dynamic programming

Status : Correct ***Marks : 1/1***

vtu30672

3. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct ***Marks : 1/1***

4. Which data structure is commonly used to implement the Branch and Bound approach for solving the Knapsack problem?

Answer

Priority Queue

Status : Correct ***Marks : 1/1***

5. In the Branch and Bound approach, what does the term “bounding function” refer to?

Answer

A function that estimates the upper bound on the maximum value that can be achieved from a node.

Status : Correct ***Marks : 1/1***

6. Which strategy is commonly used in the Branch and Bound method to decide which node to explore next?

Answer

Best-first search

Status : Correct ***Marks : 1/1***

7. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Ignoring nodes with a weight exceeding the capacity.

Status : Correct ***Marks : 1/1***

8. The travelling salesman problem can be solved using _____.

Answer

a minimum spanning tree

Status : Correct ***Marks : 1/1***

9. Travelling salesman problem is an example of _____.

Answer

None of the mentioned options

Status : Correct ***Marks : 1/1***

10. How does the practical Travelling salesman problem differ from the classical Travelling salesman problem?

Answer

In the practical travelling salesman problem, each vertex can be visited more than once.

Status : Correct ***Marks : 1/1***

11. The traveling salesman problem involves n cities with paths connecting the cities. The time taken for traversing through all the cities without knowing in advance the length of a minimum tour is

Answer

O(n!)

Status : Correct ***Marks : 1/1***

12. In which search problem, to find the shortest path, each city must be

visited only once?

Answer

Travelling Salesman problem

Status : Correct ***Marks : 1/1***

13. What is the main goal of using Branch and Bound in TSP?

Answer

To minimize search space by pruning unpromising paths

Status : Correct ***Marks : 1/1***

14. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Bound (lower bound of cost)

Status : Correct ***Marks : 1/1***

15. Given items with weights [10, 20, 30] and values [60, 100, 120], what is the value-to-weight ratio for the second item?

Answer

5

Status : Correct ***Marks : 1/1***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 4_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

You are a delivery manager who needs to plan the delivery route for a fleet of trucks to a set of distribution centers in a city. Each distribution center is at a different location, and the cost of traveling between centers is known.

Your goal is to find the shortest route that allows a truck to visit each distribution center exactly once and return to the starting point, using the Traveling Salesman Problem (TSP). Write a program to calculate the minimum cost of this route and output the optimal path.

Note: Solve using Branch and Bound approach.

Input Format

The first line contains an integer n, the number of distribution centers.

The next n lines contain n space-separated integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from distribution center i to distribution center j.

Output Format

The first line prints:

Minimum Cost is: X

The second line prints:

Optimal Path: followed by the sequence of visited centers ending at the start node

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 3 9 5
3 0 6 4
9 6 0 7
5 4 7 0

Output: Minimum Cost is: 21
Optimal Path: 0 1 2 3 0

Answer

```
#include <stdio.h>
#include <limits.h>

int n;
```

vtu30672

```
int adj[10][10];
int visited[10];
int minCost = INT_MAX;
int path[10], bestPath[10];
void copyPath(int level) {
    for (int i = 0; i <= level; i++)
        bestPath[i] = path[i];
}
void tsp(int level, int costSoFar) {
    if (level == n) {
        int totalCost = costSoFar + adj[path[level-1]][path[0]];
        if (totalCost < minCost) {
            minCost = totalCost;
            copyPath(level);
            bestPath[n] = path[0];
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            visited[i] = 1;
            path[level] = i;
            tsp(level + 1, costSoFar + (level == 0 ? 0 : adj[path[level-1]][i]));
            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);
    for (int i = 0; i < n; i++) visited[i] = 0;
    path[0] = 0;
    visited[0] = 1;
    tsp(1, 0);
    printf("Minimum Cost is: %d\n", minCost);
    printf("Optimal Path: ");
    for (int i = 0; i <= n; i++)
        printf("%d%s", bestPath[i], i == n ? "\n" : " ");
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n, the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from gallery i to gallery j.

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20

10 0 35 25
15 35 0 30
20 25 30 0

Output: Minimum Cost is: 80

Answer

```
#include <stdio.h>
#include <limits.h>

int n;
int adj[10][10];
int visited[10];
int minCost = INT_MAX;
void tsp(int pos, int level, int costSoFar) {
    if (level == n) {
        int totalCost = costSoFar + adj[pos][0];
        if (totalCost < minCost)
            minCost = totalCost;
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            visited[i] = 1;
            tsp(i, level + 1, costSoFar + adj[pos][i]);
            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);
    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    tsp(0, 1, 0);
    printf("Minimum Cost is: %d\n", minCost);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Sarah, a young entrepreneur starting her e-commerce business, has a limited budget for storing products in her warehouse. Each product has a specific weight and value, and Sarah aims to maximize the total value of the products in her warehouse without exceeding the weight capacity.

To achieve this, Sarah decided to apply the dynamic programming technique, which helps explore potential product combinations and avoid unnecessary calculations.

Input Format

The first line contains a single integer N, representing the number of products.

The next N lines each contain two integers: weight[i] and value[i], where weight[i] is the weight of the product and value[i] is the value (profit) of the product.

The next line contains a single integer W, representing the weight capacity of the warehouse.

Output Format

The output prints a single integer, representing the maximum total profit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5
2 40
3 50
1 100
5 95

3 30
10
Output: 245

Answer

```
#include <stdio.h>

int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    int N;
    scanf("%d", &N);

    int weight[N], value[N];
    for (int i = 0; i < N; i++)
        scanf("%d %d", &weight[i], &value[i]);

    int W;
    scanf("%d", &W);
    int dp[N+1][W+1];
    for (int i = 0; i <= N; i++)
        for (int w = 0; w <= W; w++)
            dp[i][w] = 0;
    for (int i = 1; i <= N; i++) {
        for (int w = 0; w <= W; w++) {
            if (weight[i-1] <= w)
                dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight[i-1]] + value[i-1]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }

    printf("%d\n", dp[N][W]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

You are a savvy thief preparing to rob a jewelry store. You have a knapsack with a limited capacity and a list of items, each with a certain weight and a corresponding profit.

Your objective is to maximize the total profit you can obtain by selecting items to place in your knapsack while staying within its weight capacity. Write a program to help you calculate the maximum profit you can achieve.

Note: Use the 0/1 knapsack method to solve the program.

Example

Input:

3

60 10

100 20

120 30

50

Output:

220

Explanation:

By selecting the second and third items, you achieve a maximum profit of Rs. 220. The sum of their profits (100 + 120) gives you the highest total profit while not exceeding the knapsack's weight capacity of 50.

Input Format

The first line contains an integer N, representing the number of items available in the store.

Each of the next N lines contains two space-separated integers P and W, where P is the profit that can be obtained by stealing the item and W is the weight of the item.

vtu30672

The last line contains an integer C representing the weight capacity of your knapsack.

Output Format

The output displays an integer representing the maximum total profit that you can achieve by selecting items for your knapsack.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3
60 10
100 20
120 30
50

Output: 220

Answer

```
#include <stdio.h>

int max(int a, int b) {
   return a > b ? a : b;
}

int main() {
   int N;
   scanf("%d", &N);

   int profit[N], weight[N];
   for (int i = 0; i < N; i++)
      scanf("%d %d", &profit[i], &weight[i]);

   int C;
   scanf("%d", &C);
   int dp[N+1][C+1];
   for (int i = 0; i <= N; i++)
      for (int w = 0; w <= C; w++)
         dp[i][w] = 0;
```

```
    for (int i = 1; i <= N; i++) {
        for (int w = 0; w <= C; w++) {
            if (weight[i-1] <= w)
                dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight[i-1]] + profit[i-1]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }

    printf("%d\n", dp[N][C]);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 4_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items whose total weight is exactly equal to a required target value. Each item can be selected at most once.

Given a list of item weights and a target weight K, determine whether it is possible to select a subset of items such that their sum is exactly K.

If such a subset exists, print any one valid subset.If no such subset exists, print -1.

This problem is derived from the Subset Sum problem, which is NP-Complete, and the solution must be implemented using Dynamic Programming.

Input Format

vtu30672

The first line contains two integers n and K

The second line contains n space-separated integers

Output Format

Print elements of one valid subset whose sum equals K

If no such subset exists, print -1

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 9
3 34 4 12 5

Output: 5 4

Answer

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, K;
    scanf("%d %d", &n, &K);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bool dp[n+1][K+1];

    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= K; j++)
            dp[i][j] = false;

    for (int i = 0; i <= n; i++)
        dp[i][0] = true;


    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= K; j++) {
            if (arr[i-1] <= j)
                dp[i][j] = dp[i-1][j] || dp[i-1][j - arr[i-1]];
            else
                dp[i][j] = dp[i-1][j];
        }
    }

    if (!dp[n][K]) {
        printf("-1\n");
        return 0;
    }

    int sum = K;
    int subset[n];
    int idx = 0;
    int i = n;

    while (i > 0 && sum > 0) {
        if (!dp[i-1][sum]) {

            subset[idx++] = arr[i-1];
            sum -= arr[i-1];
        }
        i--;
    }

    // Print the subset
    for (int j = 0; j < idx; j++)
        printf("%d ", subset[j]);
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Rohit is a travel planner who must organize a business trip covering multiple cities.

The travel cost between each pair of cities is given in a matrix.

He must:

Start from city 0Visit every city exactly onceReturn back to city 0Your task is to compute:

The minimum possible travel costThe sequence of cities visitedThe average cost per step (excluding the final return step)

This problem is a classic Travelling Salesman Problem (TSP) solved using

Dynamic Programming with Bitmasking (Held–Karp Algorithm).

Input Format

- The first line contains an integer n, the number of cities.
- The next n lines contain n space-separated integers, representing the cost matrix.

cost[i][j] is the cost of traveling from city i to city j.

Output Format

Print the path:

The Path is: 0--->...--->0

Print the minimum cost:

Minimum cost is X

Print the average cost per step:

Average cost per step is Y

vtu30672

Rounded to 2 decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Output: The Path is: 0--->2--->3--->1--->0
Minimum cost is 80
Average cost per step is 26.67

Answer

```
#include <stdio.h>
#include <limits.h>
#define MAXN 10
#define INF 1000000000

int n;
int cost[MAXN][MAXN];
int dp[1 << MAXN][MAXN];
int parent[1 << MAXN][MAXN];
int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    int VISITED_ALL = (1 << n);
        for (int i = 0; i < VISITED_ALL; i++)
        for (int j = 0; j < n; j++) {
            dp[i][j] = INF;
            parent[i][j] = -1;
        }

    dp[1][0] = 0;
```

```
for (int mask = 1; mask < VISITED_ALL; mask++) {
    for (int u = 0; u < n; u++) {
        if (!(mask & (1 << u))) continue;

        if (dp[mask][u] == INF) continue;

        for (int v = 0; v < n; v++) {
            if (mask & (1 << v)) continue;

            int newMask = mask | (1 << v);
            int newCost = dp[mask][u] + cost[u][v];

            if (newCost < dp[newMask][v]) {
                dp[newMask][v] = newCost;
                parent[newMask][v] = u;
            }
        }
    }
}
int minCost = INF;
int lastCity = -1;

for (int i = 1; i < n; i++) {
    if (dp[VISITED_ALL - 1][i] == INF) continue;

    int tourCost = dp[VISITED_ALL - 1][i] + cost[i][0];

    if (tourCost < minCost) {
        minCost = tourCost;
        lastCity = i;
    }
}
int path[MAXN + 1];
int mask = VISITED_ALL - 1;
int curr = lastCity;

for (int i = n - 1; i >= 1; i--) {
    path[i] = curr;
    int prev = parent[mask][curr];
    mask ^= (1 << curr);
    curr = prev;
}
```

```
    path[0] = 0;
    path[n] = 0;
    printf("The Path is: ");
    for (int i = 0; i <= n; i++) {
        printf("%d", path[i]);
        if (i != n)
            printf("--->");
    }
    printf("\n");
    printf("Minimum cost is %d\n", minCost);
    double avg = (double)minCost / (n - 1);
    printf("Average cost per step is %.2f\n", avg);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W. Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i-th integer represents the weight

of the i-th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180

Output: 280

Answer

```
#include <stdio.h>
#include <stdlib.h>
#define MAXN 105
typedef struct {
   int weight, value;
   float ratio;
} Item;

typedef struct {
   int level, profit, weight;
   float bound;
} Node;

int n, W;
Item items[MAXN];
int cmp(const void *a, const void *b) {
   Item *i1 = (Item *)a;
   Item *i2 = (Item *)b;
   if (i2->ratio > i1->ratio)
      return 1;
   else
```

```
        return -1;
    }
    float bound(Node u) {
        if (u.weight >= W)
            return 0;

        float profit_bound = u.profit;
        int j = u.level + 1;
        int totalWeight = u.weight;
        while (j < n && totalWeight + items[j].weight <= W) {
            totalWeight += items[j].weight;
            profit_bound += items[j].value;
            j++;
        }
        if (j < n) {
            profit_bound += (W - totalWeight) * items[j].ratio;
        }

        return profit_bound;
    }

    int max(int a, int b) {
        return (a > b) ? a : b;
    }

    int knapsack() {
        Node queue[10000];
        int front = 0, rear = 0;

        Node u, v;
        u.level = -1;
        u.profit = 0;
        u.weight = 0;
        u.bound = bound(u);

        queue[rear++] = u;

        int maxProfit = 0;

        while (front < rear) {
            u = queue[front++];
```

```
        if (u.level == n - 1)
            continue;
        v.level = u.level + 1;
        v.weight = u.weight + items[v.level].weight;
        v.profit = u.profit + items[v.level].value;

        if (v.weight <= W && v.profit > maxProfit)
            maxProfit = v.profit;

        v.bound = bound(v);
        if (v.bound > maxProfit)
            queue[rear++] = v;
        v.weight = u.weight;
        v.profit = u.profit;
        v.bound = bound(v);

        if (v.bound > maxProfit)
            queue[rear++] = v;
    }

    return maxProfit;
}

int main() {
    scanf("%d %d", &n, &W);

    for (int i = 0; i < n; i++)
        scanf("%d", &items[i].weight);

    for (int i = 0; i < n; i++) {
        scanf("%d", &items[i].value);
        items[i].ratio = (float)items[i].value / items[i].weight;
    }
    qsort(items, n, sizeof(Item), cmp);

    int result = knapsack();

    printf("%d\n", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Josh is learning about the Fractional Knapsack problem, where a knapsack has a fixed maximum capacity. Given a set of items, each with a value and a weight, Josh wants to maximize the total value placed into the knapsack.

Unlike the 0/1 Knapsack problem, fractions of items are allowed. Items must be selected using a Greedy strategy based on their value-to-weight ratio.

Write a program to determine the fraction of each item that should be included in the knapsack to achieve the maximum total value.

Formula

Value-to-Weight Ratio

ratio = value / weight

Fraction Calculation

fraction = remainingCapacity / weight

Notes

Items must be selected using the Greedy approach based on value-to-weight ratio.Sorting is required to determine the optimal order.Output fractions must be printed in the original item order.Use double datatype only for calculations.

Input Format

The first line contains an integer n, representing the number of items.

The second line contains n space-separated integers representing item values.

The third line contains n space-separated integers representing item weights.

The fourth line contains an integer capacity, representing knapsack capacity.

Output Format

Print n space-separated double values, representing the fraction of each item selected.

Output must be printed rounded to exactly two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
3 5 1 2 4
40 50 20 10 30
75
Output: 0.00 0.70 0.00 1.00 1.00

Answer

```
#include <stdio.h>
#include <stdlib.h>
#define MAXN 10
typedef struct {
   double value, weight, ratio;
   int index;
} Item;
int cmp(const void *a, const void *b) {
   Item *i1 = (Item *)a;
   Item *i2 = (Item *)b;

   if (i2->ratio > i1->ratio)
      return 1;
   else if (i2->ratio < i1->ratio)
      return -1;
   else
      return 0;
}

int main() {
   int n;
   scanf("%d", &n);
```

```
    Item items[MAXN];
    for (int i = 0; i < n; i++)
        scanf("%lf", &items[i].value);
    for (int i = 0; i < n; i++) {
        scanf("%lf", &items[i].weight);
        items[i].ratio = items[i].value / items[i].weight;
        items[i].index = i;
    }

    double capacity;
    scanf("%lf", &capacity);
    qsort(items, n, sizeof(Item), cmp);

    double fraction[MAXN] = {0.0};
    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {
            fraction[items[i].index] = 1.0;
            capacity -= items[i].weight;
        } else {
            fraction[items[i].index] = capacity / items[i].weight;
            capacity = 0;
            break;
        }
    }
    for (int i = 0; i < n; i++) {
        printf("%.2lf", fraction[i]);
        if (i != n - 1)
            printf(" ");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Ananya is preparing for a long trek and has a bag with limited capacity. She has several items, each with a weight and value, and she wants to maximize the total value she can carry. She may take whole items or fractions of them to fit in her bag.

The maximum value is calculated using the formula:

Maximum Value = Σ (value[i] * fraction_taken[i])

where fraction_taken[i] = min(1, remaining_capacity / weight[i])

Write a program to help Ananya determine the maximum achievable value.

Input Format

The input consists of several lines, each representing an item.

Each line (until -1) contains two integers: weight and value of an item.

The input terminates with -1.

The final line contains one integer representing the bag's capacity.

Output Format

The output prints "The maximum value of the current list is:"

On the next line, print the maximum value achievable with exactly two decimal places

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 10 60
20 100
30 120
-1
50

Output: The maximum value of the current list is:
240.00

Answer

```
#include <stdio.h>
#include <stdlib.h>
#define MAXN 1000
```

```
typedef struct {
    double weight, value, ratio;
} Item;
int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;

    if (i2->ratio > i1->ratio)
        return 1;
    else if (i2->ratio < i1->ratio)
        return -1;
    else
        return 0;
}

int main() {
    Item items[MAXN];
    int n = 0;
    while (1) {
        int w;
        scanf("%d", &w);

        if (w == -1)
            break;

        int v;
        scanf("%d", &v);

        items[n].weight = w;
        items[n].value = v;
        items[n].ratio = (double)v / w;
        n++;
    }

    double capacity;
    scanf("%lf", &capacity);
    qsort(items, n, sizeof(Item), cmp);
    double totalValue = 0.0;
    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {
            totalValue += items[i].value;
            capacity -= items[i].weight;
```

```
        } else {
            totalValue += items[i].ratio * capacity;
            break;
        }
    }
    printf("The maximum value of the current list is:\n");
    printf("%.2lf\n", totalValue);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 4_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48.5

Section 1 : Coding

1. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

Input Format

vtu30672

The first line of input consists of an integer n, representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j.

Output Format

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 4 1 3
4 0 2 1
1 2 0 5
3 1 5 0
Output: The Path is: 1--->3--->2--->4--->1
Minimum cost is 7

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
#define MAXN 10
int n;
int cost[MAXN][MAXN];
bool visited[MAXN];

int minCost = INT_MAX;
int bestPath[MAXN + 1];
int currPath[MAXN + 1];
int bound(int level, int currCost) {
    int estimate = currCost;
```

```
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            int minEdge = INT_MAX;
            for (int j = 0; j < n; j++) {
                if (i != j && (!visited[j] || j == 0)) {
                    if (cost[i][j] < minEdge)
                        minEdge = cost[i][j];
                }
            }
            estimate += minEdge;
        }
    }
    return estimate;
}

void tsp(int level, int currCost) {
    if (level == n) {
        if (cost[currPath[level - 1]][0] != 0) {
            int totalCost = currCost + cost[currPath[level - 1]][0];

            if (totalCost < minCost) {
                minCost = totalCost;

                for (int i = 0; i < n; i++)
                    bestPath[i] = currPath[i];

                bestPath[n] = 0;
            }
        }
        return;
    }

    for (int i = 1; i < n; i++) {
        if (!visited[i]) {
            int nextCost = currCost + cost[currPath[level - 1]][i];

            visited[i] = true;
            currPath[level] = i;
            if (bound(level + 1, nextCost) < minCost) {
                tsp(level + 1, nextCost);
            }
```

```
            visited[i] = false;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    for (int i = 0; i < n; i++)
        visited[i] = false;

    visited[0] = true;
    currPath[0] = 0;

    tsp(1, 0);
    printf("The Path is: ");
    for (int i = 0; i <= n; i++) {
        printf("%d", bestPath[i] + 1);
        if (i != n)
            printf("--->");
    }
    printf("\n");

    printf("Minimum cost is %d\n", minCost);

    return 0;
}
```

Status : Partially correct **Marks : 8.5/10**

2. Problem Statement

Karan is planning a delivery route that starts from city 0, visits every other city exactly once, and returns back to city 0.

The travel costs between cities are given as a cost matrix.

Your task is to find the minimum possible travel cost and print the optimal route.

Cost Formula

For a route:

0 a b ... 0

Total Cost = cost[0] [a] + cost[a] [b] + ... + cost[last] [0]

Input Format

The first line contains an integer n, representing the number of cities.

The next n lines contain n space-separated integers representing the cost matrix.

Output Format

The output prints multiple lines showing path exploration in the format:

The first line prints the minimum travel cost.

The second line prints the optimal path starting from city 0 and ending at city 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Output: 80
0 1 3 2 0

vtu30672

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
#define MAXN 5
int n;
int cost[MAXN][MAXN];
bool visited[MAXN];

int minCost = INT_MAX;
int bestPath[MAXN + 1];
int currPath[MAXN + 1];


void tsp(int level, int currCost) {

    if (level == n) {
        int totalCost = currCost + cost[currPath[level - 1]][0];

        if (totalCost < minCost) {
            minCost = totalCost;

            for (int i = 0; i < n; i++)
                bestPath[i] = currPath[i];

            bestPath[n] = 0;
        }
        return;
    }

    for (int i = 1; i < n; i++) {
        if (!visited[i]) {
            visited[i] = true;
            currPath[level] = i;

            tsp(level + 1, currCost + cost[currPath[level - 1]][i]);

            visited[i] = false;
        }
    }
}
```

```
int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    for (int i = 0; i < n; i++)
        visited[i] = false;

    visited[0] = true;
    currPath[0] = 0;

    tsp(1, 0);
    printf("%d\n", minCost);

    for (int i = 0; i <= n; i++) {
        printf("%d", bestPath[i]);
        if (i != n)
            printf(" ");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n, the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The ith integer on the jth line represents the cost of traveling from location i to location j.

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 10 8 9 7
10 0 10 5 6
8 10 0 8 9
9 5 8 0 6
7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1
Minimum cost is 34

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
#define MAXN 10
int n;
int cost[MAXN][MAXN];
bool visited[MAXN];
int minCost = INT_MAX;
int bestPath[MAXN + 1];
int currPath[MAXN + 1];
void tsp(int level, int currCost) {

    if (level == n) {
```

```
        int totalCost = currCost + cost[currPath[level - 1]][0];

        if (totalCost < minCost) {
            minCost = totalCost;

            for (int i = 0; i < n; i++)
                bestPath[i] = currPath[i];

            bestPath[n] = 0;
        }
        return;
    }

    for (int i = 1; i < n; i++) {
        if (!visited[i]) {

            int nextCost = currCost + cost[currPath[level - 1]][i];
            if (nextCost >= minCost)
                continue;

            visited[i] = true;
            currPath[level] = i;

            tsp(level + 1, nextCost);

            visited[i] = false;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    for (int i = 0; i < n; i++)
        visited[i] = false;

    visited[0] = true;
    currPath[0] = 0;
```

```
    tsp(1, 0);
    printf("Shortest Path: ");
    for (int i = 0; i <= n; i++) {
        printf("%d", bestPath[i] + 1);
        if (i != n)
            printf(" ");
    }
    printf("\n");

    printf("Minimum cost is %d\n", minCost);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Michael has been given an important task—developing a program to solve the classic 0/1 Knapsack Problem. His goal is to help users select the most valuable combination of items while staying within a specified weight limit.

In this problem, each item has a value and a weight, and the objective is to maximize the total value without exceeding the given weight capacity.

Michael's program should determine the maximum possible value that can be obtained by selecting the optimal set of items.

Input Format

The first line of input consists of an integer n, representing the number of items available for selection.

The second line of input consists of n space-separated integers, P, which represent the values of the items.

The third line of input consists of n space-separated integers, C, which represent the weights of the corresponding items.

The fourth line of input consists of an integer W, representing the maximum

weight limit of the knapsack.

Output Format

The output displays the maximum value that can be obtained using the knapsack algorithm in the following format: "Maximum value: [maximum value]"

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 6
300 150 120 100 90 80
6 3 3 2 2 2
10
Output: Maximum value: 490

Answer

```
#include <stdio.h>
#define MAXN 20
#define MAXW 1005
int main() {
    int n;
    scanf("%d", &n);
    int value[MAXN], weight[MAXN];

    for (int i = 0; i < n; i++)
        scanf("%d", &value[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &weight[i]);

    int W;
    scanf("%d", &W);

    int dp[MAXN][MAXW];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (weight[i - 1] <= w) {
```

```
            int include = value[i - 1] + dp[i - 1][w - weight[i - 1]];
            int exclude = dp[i - 1][w];
            dp[i][w] = (include > exclude) ? include : exclude;
        } else {
            dp[i][w] = dp[i - 1][w];
        }
    }
  }

  printf("Maximum value: %d\n", dp[n][W]);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Ragul is interested in solving the knapsack problem. He has a list of n items, each with a weight and a value. Ragul wants to determine the maximum value he can obtain by selecting a combination of items to fit into his knapsack, which has a maximum weight capacity.

Help Ragul write a program that calculates the following:

Calculate and print the sum of all n values.Calculate and print the average value of all n values with exactly 2 decimal places.Calculate and print the maximum value that can be obtained by selecting a combination of items to fit into the knapsack without exceeding its capacity.

Note: Use the 0/1 knapsack method to solve the problem.

Input Format

The first line of input contains an integer n, representing the number of items.

The next line contains n space-separated integers, representing the weights of the items.

The next line contains n space-separated integers, representing the values of the items.

The last line contains an integer C, representing the maximum weight capacity of the knapsack.

Output Format

The output consists of the following in each line:

1. The sum of all n values as an integer.
2. The average value of all n values with exactly 2 decimal places is a double value.
3. The maximum value that can be obtained as an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 15 20
45 65 70
30

Output: Sum of values: 180
Average of values: 60.00
Maximum amount: 115

Answer

```
#include <stdio.h>
#define MAXN 10
#define MAXC 30
int main() {
    int n;
    scanf("%d", &n);

    int weight[MAXN], value[MAXN];

    for (int i = 0; i < n; i++)
        scanf("%d", &weight[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &value[i]);
```

```
    int C;
    scanf("%d", &C);
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += value[i];
    double avg = (double)sum / n;
    int dp[MAXC + 1] = {0};

    for (int i = 0; i < n; i++) {
        for (int w = C; w >= weight[i]; w--) {
            int include = value[i] + dp[w - weight[i]];
            if (include > dp[w])
                dp[w] = include;
        }
    }
    printf("Sum of values: %d\n", sum);
    printf("Average of values: %.2f\n", avg);
    printf("Maximum amount: %d\n", dp[C]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct ***Marks : 1/1***

2. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct ***Marks : 1/1***

vtu30672

3. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Smallest element

Status : Wrong ***Marks : 0/1***

4. What is the worst-case time complexity of the heap sort algorithm?

Answer

O(n log n)

Status : Correct ***Marks : 1/1***

5. In a quick sort algorithm, where are the smaller elements placed to the pivot during the partition process, assuming we are sorting in increasing order?

Answer

To the left of the pivot

Status : Correct ***Marks : 1/1***

6. Which of the following is true about Quicksort?

Answer

It is an in-place sorting algorithm

Status : Correct ***Marks : 1/1***

7. What is the worst-case time complexity of the Quicksort algorithm?

Answer

$O(n^2)$

Status : Correct ***Marks : 1/1***

vtu30672

8. Given a sorted array of 1000 unique integers, how many maximum comparisons does Binary Search make in the worst case?

Answer

10

Status : Correct ***Marks : 1/1***

9. Suppose you are using Binary Search on a sorted array of even length (e.g., 8 elements). At each step, you calculate the middle as:

mid = (low + high) // 2

Which of the following statements is always true?

Answer

The mid index may not divide the array into equal halves in all steps.

Status : Correct ***Marks : 1/1***

10. You apply Binary Search on the sorted array [3, 7, 10, 15, 21, 30] to find the element 13. Which sequence of middle elements will Binary Search compare with before declaring that the element is not present?

Answer

10 21 15

Status : Correct ***Marks : 1/1***

11. You are using Binary Search to find the first occurrence of a value x in a sorted array that may contain duplicates. Which of the following modifications is necessary?

Answer

When x == arr[mid], move high to mid - 1.

Status : Correct ***Marks : 1/1***

vtu30672

12. A student implements Binary Search and calculates the middle index as:

mid = (low + high) // 2

But when the input values of low and high are both large (e.g., low = 2_000_000_000, high = 2_000_000_010), the program crashes due to integer overflow. What is the best fix?

Answer

Use mid = low + (high - low) // 2

Status : Correct ***Marks : 1/1***

13. Is Merge Sort a stable sorting algorithm?

Answer

Yes, always stable.

Status : Correct ***Marks : 1/1***

14. Merge sort is _____.

Answer

Comparison-based sorting algorithm

Status : Correct ***Marks : 1/1***

15. Which of the following methods is used for sorting in merge sort?

Answer

merging

Status : Correct ***Marks : 1/1***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 5_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

Input Format

The first line of input consists of an integer n, denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

Output Format

vtu30672

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
4 1 5 3 2 6

Output: Given Array
4 1 5 3 2 6
After iteration 1
1 4 5 3 2 6
After iteration 2
1 4 5 3 2 6
After iteration 3
1 4 5 2 3 6
After iteration 4

vtu30672

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

Answer

```
#include <stdio.h>
int iteration = 1;
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
void merge(int arr[], int left, int mid, int right, int n) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
```

```
        k++;
    }
    printf("After iteration %d\n", iteration++);
    printArray(arr, n);
}
void mergeSort(int arr[], int left, int right, int n) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid, n);
        mergeSort(arr, mid + 1, right, n);

        merge(arr, left, mid, right, n);
    }
}
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Given Array\n");
    printArray(arr, n);
    mergeSort(arr, 0, n - 1, n);
    printf("Sorted Array\n");
    printArray(arr, n);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer n, representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)
- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

Output Format

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:

vtu30672

- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.
- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Harish
7.8
23
ece
Trell
8
21
cse
Kamalesh
7
25
it
Maaran
5.6
20
civil
age

Output: Sorted Student Records:
Name GPA Age Major
Maaran 5.60 20 civil
Trell 8.00 21 cse
Harish 7.80 23 ece
Kamalesh 7.00 25 it

Answer

vtu30672

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char name[50];
    float gpa;
    int age;
    char major[50];
} Student;
void swap(Student* a, Student* b) {
    Student temp = *a;
    *a = *b;
    *b = temp;
}
int compare(Student a, Student b, char criterion[]) {
    if (strcmp(criterion, "gpa") == 0) {
        return (a.gpa < b.gpa); // Sort ascending
    } else if (strcmp(criterion, "age") == 0) {
        return (a.age < b.age);
    } else if (strcmp(criterion, "major") == 0) {
        return (strcmp(a.major, b.major) < 0);
    }
    return 0;
}
int partition(Student arr[], int low, int high, char criterion[]) {
    Student pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (compare(arr[j], pivot, criterion)) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
void quickSort(Student arr[], int low, int high, char criterion[]) {
    if (low < high) {
        int pi = partition(arr, low, high, criterion);
        quickSort(arr, low, pi - 1, criterion);
        quickSort(arr, pi + 1, high, criterion);
    }
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    Student s[10];
    for (int i = 0; i < n; i++) {
        scanf("%s", s[i].name);
        scanf("%f", &s[i].gpa);
        scanf("%d", &s[i].age);
        scanf("%s", s[i].major);
    }
    char criterion[10];
    scanf("%s", criterion);
    quickSort(s, 0, n - 1, criterion);
    printf("Sorted Student Records:\n");
    printf("Name\t\tGPA\t\tAge\t\tMajor\n");
    for (int i = 0; i < n; i++) {
        printf("%-10s\t%.2f\t%-3d \t%-15s\n", s[i].name, s[i].gpa, s[i].age, s[i].major);
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10
banana pineapple mango chikoo jack star guava lemon tomato orange
Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
#include <stdio.h>
#include <string.h>
void heapify(char arr[][100], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;
    if (right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;
    if (largest != i) {
        char temp[100];
        strcpy(temp, arr[i]);
        strcpy(arr[i], arr[largest]);
        strcpy(arr[largest], temp);
        heapify(arr, n, largest);
    }
}
void heapSort(char arr[][100], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        char temp[100];
        strcpy(temp, arr[0]);
        strcpy(arr[0], arr[i]);
        strcpy(arr[i], temp);
        heapify(arr, i, 0);
    }
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    char words[n][100];
    for (int i = 0; i < n; i++) {
        scanf("%s", words[i]);
    }

    heapSort(words, n);
    for (int i = 0; i < n; i++) {
        printf("%s ", words[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8

8

Output:
7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

Input Format

The first line of input consists of an integer n, representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x, representing the specific book ID to find.

Output Format

The output displays indexes of the first and last occurrence of x, as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
1 2 2 2 2 3 4 8 8 8
8
Output: 7 9

Answer

```
#include <stdio.h>
int findFirst(int arr[], int low, int high, int x, int res) {
   if (low > high) {
      return res;
   }
   int mid = low + (high - low) / 2;
   if (arr[mid] == x) {
      res = mid;
```

```
        return findFirst(arr, low, mid - 1, x, res);
    } else if (x < arr[mid]) {
        return findFirst(arr, low, mid - 1, x, res);
    } else {
        return findFirst(arr, mid + 1, high, x, res);
    }
}
int findLast(int arr[], int low, int high, int x, int res) {
    if (low > high) {
        return res;
    }
    int mid = low + (high - low) / 2;
    if (arr[mid] == x) {
        res = mid;
        return findLast(arr, mid + 1, high, x, res);
    } else if (x < arr[mid]) {
        return findLast(arr, low, mid - 1, x, res);
    } else {
        return findLast(arr, mid + 1, high, x, res);
    }
}

int main() {
    int n, x;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    scanf("%d", &x);
    int first = findFirst(arr, 0, n - 1, x, -1);
    int last = findLast(arr, 0, n - 1, x, -1);

    if (first == -1) {
        printf("NO OCCURRENCES\n");
    } else {
        printf("%d %d\n", first, last);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 5_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers, count the number of smaller elements that appear to the right of each element. Implement a solution using merge sort to count these elements efficiently. Output the result as an array of counts corresponding to each element in the original array.

Example

Input:

4

5 2 6 1

Output:

2 1 1 0

vtu30672

Explanation:

To the right of 5, there are 2 smaller elements (2 and 1).

To the right of 2, there is only 1 smaller element (1).

To the right of 6, there is 1 smaller element (1).

To the right of 1, there is 0 smaller element.

Input Format

The first line of input contains a single integer n, representing the length of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single line containing the counts of smaller elements for each value in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
5 2 6 1
Output: 2 1 1 0

Answer

```
#include <stdio.h>
#include <stdlib.h>

int counts[20];

void merge(int arr[], int indices[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int left_indices[n1], right_indices[n2];
```

```
    for (int i = 0; i < n1; i++)
        left_indices[i] = indices[left + i];
    for (int j = 0; j < n2; j++)
        right_indices[j] = indices[mid + 1 + j];

    int i = 0, j = 0, k = left;
    int right_smaller_count = 0;

    while (i < n1 && j < n2) {

        if (arr[right_indices[j]] < arr[left_indices[i]]) {
            indices[k] = right_indices[j];
            right_smaller_count++;
            j++;
        } else {

            indices[k] = left_indices[i];

            counts[left_indices[i]] += right_smaller_count;
            i++;
        }
        k++;
    }

    while (i < n1) {
        indices[k] = left_indices[i];
        counts[left_indices[i]] += right_smaller_count;
        i++;
        k++;
    }
    while (j < n2) {
        indices[k] = right_indices[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int indices[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, indices, left, mid);
```

```
        mergeSort(arr, indices, mid + 1, right);
        merge(arr, indices, left, mid, right);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    int indices[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        indices[i] = i;
        counts[i] = 0;
    }

    mergeSort(arr, indices, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d ", counts[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Ravi, a mathematics teacher, is helping his students understand quadratic equations. He wants to apply the quadratic formula [A * x2 + B * x + C] to each number in a sorted array. Once the formula is applied, the modified numbers need to be sorted again. Ravi plans to use QuickSort for efficient sorting. Help Ravi by implementing a program that performs these operations.

Input Format

The first line contains an integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array in ascending order.

The third line contains three integers A, B, and C, the coefficients of the quadratic formula.

Output Format

The first line of output prints: Array after applying the equation and sorting

The subsequent lines print: <sorted_value>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
-1 0 1 2 3 4
-1 2 -1
Output: Array after applying the equation and sorting:
-9 -4 -4 -1 -1 0

Answer

```
#include <stdio.h>

void swap(long long* a, long long* b) {
    long long t = *a;
    *a = *b;
    *b = t;
}

int partition(long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
```

```
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(long long arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    long long arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    int A, B, C;
    scanf("%d %d %d", &A, &B, &C);

    for (int i = 0; i < n; i++) {
        long long x = arr[i];
        arr[i] = (long long)A * x * x + (long long)B * x + C;
    }

    quickSort(arr, 0, n - 1);

    printf("Array after applying the equation and sorting:\n");
    for (int i = 0; i < n; i++) {
        printf("%lld ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2
Output: 20

Answer

```
#include <stdio.h>

void swap(int *a, int *b) {
   int temp = *a;
   *a = *b;
```

```
    *b = temp;
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int k;
    scanf("%d", &k);

    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    for (int i = 0; i < k - 1; i++) {
        swap(&arr[0], &arr[n - 1 - i]);
        heapify(arr, n - 1 - i, 0);
    }

    printf("%d\n", arr[0]);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Tom is working in a warehouse inventory system, where the item IDs are assigned sequentially in ascending order.

He wants to develop a program using recursive binary search to efficiently determine the closest item ID that is less than or equal to a given target ID.

The program takes the total number of items and their sorted IDs as input, assisting warehouse staff in quickly identifying the closest available item less than or equal to the target ID. Help Tom in this program.

Input Format

The first line of input consists of an integer N, representing the number of items in the warehouse.

The second line consists of N space-separated integers, representing the sorted list of item IDs.

The third line consists of an integer representing the target item ID.

Output Format

The output prints "The closest item ID less than or equal to X is Y", where X is the target ID and Y is the closest item ID less than or equal to X.

If no such element exists, print "No closest item with an ID less than or equal to X exists in the warehouse", where X is the target ID.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
17 25 38 47 51 62 79
50
Output: The closest item ID less than or equal to 50 is 47

Answer

```
#include <stdio.h>

int findFloor(int arr[], int low, int high, int target, int res) {
    if (low > high) {
        return res;
    }

    int mid = low + (high - low) / 2;

    if (arr[mid] == target) {
        return arr[mid];
    } else if (arr[mid] < target) {

        return findFloor(arr, mid + 1, high, target, arr[mid]);
    } else {

        return findFloor(arr, low, mid - 1, target, res);
    }
}

int main() {
    int n, target;

    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (scanf("%d", &target) != 1) return 0;

    int result = findFloor(arr, 0, n - 1, target, -1);

    if (result == -1) {
        printf("No closest item with an ID less than or equal to %d exists in the
```

```
warehouse\n", target);
    } else {
        printf("The closest item ID less than or equal to %d is %d\n", target, result);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Rahul owns a bookstore and maintains a sorted list of ISBN numbers of books in his inventory. When a customer asks for a specific book, Rahul wants to quickly check whether the book is available in his inventory.

Write a program that implements a non-recursive binary search to determine if a given ISBN exists in the sorted list of ISBN numbers.

Input Format

The first line contains an integer N, representing the number of ISBN numbers in the inventory.

The second line contains N space-separated integers representing the sorted list of ISBN numbers.

The third line contains a single integer X, the ISBN number to be searched.

Output Format

If the ISBN X is found in the inventory, print: "Book with ISBN X is available at index Y" where Y is the zero-based index of the ISBN in the list.

If the ISBN X is not found, print: "Book with ISBN X is not available in the inventory"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2 3 40 50 20
3
Output: Book with ISBN 3 is available at index 1

Answer

```
#include <stdio.h>

int main() {
    int n, x;

    if (scanf("%d", &n) != 1) return 0;

    int isbn[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &isbn[i]);
    }

    if (scanf("%d", &x) != 1) return 0;

    int low = 0;
    int high = n - 1;
    int foundIndex = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (isbn[mid] == x) {
            foundIndex = mid;
            break;
        } else if (isbn[mid] < x) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    if (foundIndex != -1) {
        printf("Book with ISBN %d is available at index %d\n", x, foundIndex);
    } else {
        printf("Book with ISBN %d is not available in the inventory\n", x);
```

```
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 2_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

In a kingdom far away, two types of soup are prepared daily — Soup A and Soup B. The royal chef always starts with n milliliters of each soup. Each time a servant is served, one of the following four serving operations is chosen at random (each with equal probability of 0.25):

Serve 100 ml of Soup A and 0 ml of Soup B

Serve 75 ml of Soup A and 25 ml of Soup B

Serve 50 ml of Soup A and 50 ml of Soup B

Serve 25 ml of Soup A and 75 ml of Soup B

The serving continues until either or both soups become empty. If the required soup amount for an operation exceeds the available quantity,

serve as much as possible (up to what remains).

You are to compute the probability that Soup A becomes empty first, plus half the probability that both soups become empty at the same time.

Special Note:

Since all operations serve soup in multiples of 25 ml, you must first scale n down to units of 25 ml before recursive processing:

Example

Sample Input 1

n = 50

Sample Output 2

0.625

Explanation

If we choose the first two operations, A will become empty first.

For the third operation, A and B will become empty at the same time.

For the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is 0.25 * (1 + 1 + 0.5 + 0) = 0.625.

Input Format

The input consists of an integer n, representing the total volume of the soup.

Output Format

The output prints the probability that soup A will be empty first, plus half the probability that A and B become empty at the same time.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

Output: 0.625

Answer

```
#include <stdio.h>
#include <string.h>
double dp[101][101];
double solve(int a, int b) {

    if (a <= 0 && b <= 0) return 0.5;
    if (a <= 0) return 1.0;
    if (b <= 0) return 0.0;
    if (dp[a][b] >= 0) return dp[a][b];
    dp[a][b] = 0.25 * (
        solve(a - 4, b) +
        solve(a - 3, b - 1) +
        solve(a - 2, b - 2) +
        solve(a - 1, b - 3)
    );

    return dp[a][b];
}

int main() {
    int n;
    scanf("%d", &n);
    int units = (n + 24) / 25;
    for (int i = 0; i <= 100; i++) {
        for (int j = 0; j <= 100; j++) {
            dp[i][j] = -1.0;
        }
    }

    double result = solve(units, units);

    printf("%.3f", result);

    return 0;
}
```

Status : Correct *Marks : 10/10*

2. Problem Statement

In a unique team-building exercise, a company has decided to distribute a certain number of resources among its departments based on mathematical powers. The company has x resources that need to be distributed such that each department receives a different power of an integer. Specifically, each department i receives n resources, where n is a given power.

George, the resource manager, needs to find all possible ways to distribute exactly x resources using the powers of integers starting from 1. Each integer's power can only be used once.

Write a program to help George determine the number of ways to distribute x resources in this manner using a recursive manner.

Example 1

Input:

10

2

Output:

1

Explanation:

x = 10

n = 2

10 = 12 + 32, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

x = 100

n = 2

100 = 102 OR 62 + 82 OR 12 + 32 + 42 + 52 + 72, hence total 3 possibilities.

Input Format

The first line of input consists of the integer x, representing the total number of resources.

The second line consists of the integer n, representing the power to which the integers are raised.

Output Format

The output should be a single integer representing the number of ways to distribute exactly x resources.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100
2

Output: 3

Answer

```
#include <stdio.h>
#include <math.h>
int countWays(int x, int n, int num) {
    int power = pow(num, n);
    if (power == x) return 1;
    if (power > x) return 0;
    return countWays(x - power, n, num + 1) +
```

```
    countWays(x, n, num + 1);
}

int main() {
    int x, n;
    scanf("%d", &x);
    scanf("%d", &n);

    int result = countWays(x, n, 1);

    printf("%d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
#include <stdio.h>
int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        if (n & 1)
            count++;
        n = n >> 1;
    }
    return count;
}
int main() {
    int n;
    scanf("%d", &n);
    printf("%d", countSetBits(n));
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given a square matrix of size n×n, rotate the elements along each layer (or boundary) of the matrix by one position in a clockwise direction. The rotation should be performed layer by layer, starting from the outermost layer and proceeding inward.

Examples:

Input 2:

3 3

1 2 3

4 5 6

7 8 9

Output 2:

4 1 2

7 5 3

8 9 6

Explanation:

For a clockwise boundary rotation by one step:

We start with the outermost layer (there's only one layer in a 3×3 matrix)

We save the top-left element (1)

Move left edge upward: 4 1, 7 4

Move bottom edge leftward: 8 7, 9 8

Move right edge downward: 6 9, 3 6

Move top edge rightward: 2 3

Place the saved element (1) in the second position of the top row, so 1 2

Input Format

The first line contains two integers R and C — the number of rows and columns in the matrix.

The next R lines each contain C integers, representing the elements of the matrix.

Output Format

The output displays the rotated matrix after performing the clockwise boundary rotation.

Refer to the sample input and output for formatting specifications.

Sample Test Case

vtu30672

Input: 4 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

Output: 5 1 2 3
9 10 6 4
13 11 7 8
14 15 16 12

Answer

```
#include <stdio.h>
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    int a[5][5];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    int top = 0, bottom = n - 1;
    int left = 0, right = m - 1;
    while (top < bottom && left < right) {
        int prev = a[top + 1][left];
        for (int i = left; i <= right; i++) {
            int temp = a[top][i];
            a[top][i] = prev;
            prev = temp;
        }
        top++;
        for (int i = top; i <= bottom; i++) {
            int temp = a[i][right];
            a[i][right] = prev;
            prev = temp;
        }
        right--;
        for (int i = right; i >= left; i--) {
            int temp = a[bottom][i];
            a[bottom][i] = prev;
            prev = temp;
        }
```

```
        bottom--;
        for (int i = bottom; i >= top; i--) {
            int temp = a[i][left];
            a[i][left] = prev;
            prev = temp;
        }
        left++;
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Given a 2D matrix of size ROW × COL, print all elements of the matrix in diagonal order.

Diagonal Order Definition:

Start from the first element at position (0,0).Traverse all diagonals starting from the leftmost column going downward, then continue with diagonals starting from the bottom row going rightward.Within each diagonal, print elements from the top-right corner to bottom-left corner.

Example:

Input matrix

5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Diagonal printing of the above matrix is

1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

Input Format

The first line of input contains two integers, ROW and COL.

The next ROW lines each contain COL space-separated integers

Output Format

The output consists of multiple lines, one for each diagonal. Each line contains the matrix elements of that diagonal in the order from top-right to bottom-left, separated by spaces.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
Output: 1
5 2

vtu30672

9 6 3
13 10 7 4
17 14 11 8
18 15 12
19 16
20

Answer

```
#include <stdio.h>

int main() {
    int ROW, COL;
    scanf("%d %d", &ROW, &COL);

    int a[10][10];
    for (int i = 0; i < ROW; i++) {
        for (int j = 0; j < COL; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    for (int start = 0; start < ROW; start++) {
        int i = start;
        int j = 0;

        while (i >= 0 && j < COL) {
            printf("%d ", a[i][j]);
            i--;
            j++;
        }
        printf("\n");
    }
    for (int start = 1; start < COL; start++) {
        int i = ROW - 1;
        int j = start;

        while (i >= 0 && j < COL) {
            printf("%d ", a[i][j]);
            i--;
            j++;
        }
        printf("\n");
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu30672

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 3_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

You are given a set of N points on a 2D plane. Your task is to find the distance between the closest pair of points.

Write a program that takes as input the number of points N, followed by the x and y coordinates of each point. The program should output the distance between the closest pair of points, rounded to two decimal places.

Input Format

The first line of input consists of an integer N, representing the number of points.

The following N lines represent the coordinate points x and y, separated by space.

Output Format

vtu30672

The output prints a single floating-point number representing the distance between the closest pair of points, rounded to two decimal places.

Sample Test Case

Input: 3
2 3
12 30
40 50

Output: 28.79

Answer

```
#include <stdio.h>
#include <math.h>
#include <float.h>
typedef struct {
    double x;
    double y;
} Point;
double calculateDistance(Point p1, Point p2) {
    return sqrt(pow(p1.x - p2.x, 2) + pow(p1.y - p2.y, 2));
}
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    Point points[n];
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }
    double minDistance = DBL_MAX;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double dist = calculateDistance(points[i], points[j]);
            if (dist < minDistance) {
                minDistance = dist;
            }
        }
    }
    printf("%.2f\n", minDistance);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Ishita is working on a mapping application that helps locate the nearest landmarks on a 2D map. Each landmark is represented by its (x, y) coordinate. Her task is to find out the shortest distance between any two landmarks from the given list.

Help Ishita write a program that calculates the distance between the closest pair of points (landmarks) on the map.

Input Format

The first line of input contains an integer N representing the number of landmarks.

The next N lines contain two space-separated integers, representing the x and y coordinates of each landmark.

Output Format

The output prints a single floating-point number representing the distance between any two landmarks rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
5 7
12 32
21 23
43 65

Output: 12.73

Answer

```
#include <stdio.h>
#include <math.h>
#include <float.h>
int main() {
   int n;
```

vtu30672

```
    if (scanf("%d", &n) != 1) {
        return 0;
    }
    int x[n], y[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &x[i], &y[i]);
    }
    double min_dist = DBL_MAX;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double dx = x[i] - x[j];
            double dy = y[i] - y[j];
            double distance = sqrt(dx * dx + dy * dy);
            if (distance < min_dist) {
                min_dist = distance;
            }
        }
    }
    printf("%.2f\n", min_dist);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N, representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y, represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 3
1 1
10 18
4 9

Output: Closest pair of points: (0, 3) and (1, 1)
Distance: 2.24

Answer

```
#include <stdio.h>
#include <math.h>
#include <float.h>
typedef struct {
    double x;
    double y;
} Point;
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    Point points[10];
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }
    double minDistance = DBL_MAX;
    Point p1, p2;
    for (int i = 0; i < n; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
            double dx = points[i].x - points[j].x;
            double dy = points[i].y - points[j].y;
            double dist = sqrt(dx * dx + dy * dy);

            if (dist < minDistance) {
                minDistance = dist;
                p1 = points[i];
                p2 = points[j];
            }
        }
    }
    printf("Closest pair of points: (%.0f, %.0f) and (%.0f, %.0f)\n", p1.x, p1.y, p2.x,
p2.y);
    printf("Distance: %.2f\n", minDistance);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ragu, a passionate computer science student, is exploring geometry problems to sharpen his algorithmic thinking. Today, he challenges himself to determine the two closest points among a set of 2D coordinates.

Ragu must find the closest pair of points from the given set and compute the Euclidean distance between them with high precision. He also wants the output to display which two points are the closest, formatted clearly.

Input Format

The first line of input contains an integer N representing the number of points in the 2D plane.

The next N lines each contain two space-separated real numbers x and y representing the coordinates of each point.

Output Format

The first line of output prints the coordinates of the two points forming the

closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
1 2
3 4
1 0
6 8
2 3
Output: Closest pair of points: (1, 2) and (2, 3)
Distance: 1.41

Answer

```
#include <stdio.h>
#include <math.h>
#include <float.h>
typedef struct {
    double x;
    double y;
} Point;
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Point points[n];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }

    double minDistance = DBL_MAX;
    Point bestP1, bestP2;
```

```
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double dx = points[i].x - points[j].x;
            double dy = points[i].y - points[j].y;
            double currentDistance = sqrt(dx * dx + dy * dy);
            if (currentDistance < minDistance) {
                minDistance = currentDistance;
                bestP1 = points[i];
                bestP2 = points[j];
            }
        }
    }

    printf("Closest pair of points: (%.0f, %.0f) and (%.0f, %.0f)\n",
            bestP1.x, bestP1.y, bestP2.x, bestP2.y);
    printf("Distance: %.2f\n", minDistance);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 5_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48

Section 1 : Coding

1. Problem Statement

Suppose you are working on a project that involves sorting a large dataset of student records based on a single criterion, such as their CGPA, age, or major. You have decided to use a modified quicksort algorithm to efficiently sort the dataset while ensuring that the sorting is stable, i.e., that records with equal values maintain their original relative order.

To achieve this, you will implement a modified version of the quicksort algorithm that maintains stability during the sorting process. Specifically, you will use a three-way partitioning scheme that sorts the records into three groups: those with values less than the pivot, those with values equal to the pivot, and those with values greater than the pivot. When the algorithm processes subarrays with equal pivot values, it uses insertion sort to maintain stability.

vtu30672

Write a function that implements this modified version of the quicksort algorithm and uses it to sort the student records according to one specified criterion (CGPA, age, or major). Your function should take as input a list of student records and return the sorted list in ascending order based on the chosen criterion. If two records have the same values for the sorting criterion, their relative order should be maintained in the sorted list.

Input Format

The first line contains an integer n, representing the number of student records to be sorted.

The next n groups of 4 lines each contain the details of each student:

- Line 1: Name (string)
- Line 2: CGPA (float)
- Line 3: Age (integer)
- Line 4: Major (string)

The last line contains the sorting criteria (one of three options: "CGPA", "age", or "major").

Output Format

The output displays the sorted student records in a table format with the following specifications:

Header row: "Name CGPA Age Major"

Each student record is displayed in one row with the following format:

- Name: Left-aligned, 10 characters wide
- CGPA: Left-aligned, 8 characters wide, displayed with 2 decimal places
- Age: Left-aligned, 5 characters wide
- Major: Left-aligned, 20 characters wide

The table is sorted in ascending order according to the chosen sorting criteria.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 4

vtu30672

harish
7.8
23
ece
trello
8
21
cse
kamalesh
7
25
it
maaran
5.6
20
civil
age

Output: Name CGPA Age Major
maaran 5.60 20 civil
trello 8.00 21 cse
harish 7.80 23 ece
kamalesh 7.00 25 it

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct {
    char name[51];
    float cgpa;
    int age;
    char major[51];
} Student;

char criterion[20];

int compare(Student a, Student b) {
    if (strcmp(criterion, "age") == 0)
        return a.age - b.age;
```

```
    if (strcmp(criterion, "cgpa") == 0 || strcmp(criterion, "gpa") == 0) {
        if (a.cgpa < b.cgpa) return -1;
        if (a.cgpa > b.cgpa) return 1;
        return 0;
    }

    // major
    return strcmp(a.major, b.major);
}

/* Stable 3-way quicksort */
void quicksort(Student arr[], int n) {
    if (n <= 1) return;

    Student pivot = arr[0];

    Student *less = (Student*)malloc(n * sizeof(Student));
    Student *equal = (Student*)malloc(n * sizeof(Student));
    Student *greater = (Student*)malloc(n * sizeof(Student));

    int l = 0, e = 0, g = 0;

    for (int i = 0; i < n; i++) {
        int cmp = compare(arr[i], pivot);

        if (cmp < 0)
            less[l++] = arr[i];
        else if (cmp > 0)
            greater[g++] = arr[i];
        else
            equal[e++] = arr[i];
    }

    quicksort(less, l);
    quicksort(greater, g);

    int idx = 0;

    for (int i = 0; i < l; i++) arr[idx++] = less[i];
    for (int i = 0; i < e; i++) arr[idx++] = equal[i];
    for (int i = 0; i < g; i++) arr[idx++] = greater[i];
```

```
    free(less);
    free(equal);
    free(greater);
}

int main() {
    int n;
    scanf("%d", &n);

    Student *arr = (Student*)malloc(n * sizeof(Student));

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i].name);
        scanf("%f", &arr[i].cgpa);
        scanf("%d", &arr[i].age);
        scanf("%s", arr[i].major);
    }

    scanf("%s", criterion);

    for (int i = 0; criterion[i]; i++)
        criterion[i] = tolower(criterion[i]);

    quicksort(arr, n);

    printf("Name      CGPA     Age   Major            \n");

    for (int i = 0; i < n; i++) {
        printf("%-10s%-8.2f%-5d%-20s\n",
               arr[i].name,
               arr[i].cgpa,
               arr[i].age,
               arr[i].major);
    }

    free(arr);
    return 0;
}
```

Status : Partially correct ***Marks : 8/10***

2. Problem Statement

Vishnu, a math enthusiast, is given a task to explore the magic of numbers. He has an array of positive integers, and his goal is to find the integer with the highest digit sum in the sorted array using the merge sort algorithm.

You have to assist Vishnu in implementing the merge sort algorithm.

Example

Input:

6

789 123 456 876 234 567

Output:

The sorted array is: 123 234 456 567 789 876

The integer with the highest digit sum is: 789

Explanation

Sorted Array Output: 123 234 456 567 789 876

The digit sum of an integer is the sum of its individual digits.

In this case, the integer 789 has the highest digit sum (7 + 8 + 9 = 24).

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the array elements.

Output Format

The first line prints "The sorted array is: " followed by the sorted array in ascending order.

The second line prints "The integer with the highest digit sum is: " followed by the integer with the highest-digit sum.

vtu30672

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
123 456 789 321 654

Output: The sorted array is: 123 321 456 654 789
The integer with the highest digit sum is: 789

Answer

```
// You are using GCC
#include <stdio.h>

int arr[10], temp[10];
int n;

int digitSum(int x) {
    int sum = 0;
    while (x > 0) {
        sum += x % 10;
        x /= 10;
    }
    return sum;
}

void merge(int low, int mid, int high) {
    int i = low, j = mid + 1, k = low;

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= high)
```

```
        temp[k++] = arr[j++];

    for (i = low; i <= high; i++)
        arr[i] = temp[i];
}

void mergeSort(int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(low, mid);
        mergeSort(mid + 1, high);
        merge(low, mid, high);
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    mergeSort(0, n - 1);

    printf("The sorted array is: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    int maxSum = -1, ans = arr[0];

    for (int i = 0; i < n; i++) {
        int sum = digitSum(arr[i]);
        if (sum > maxSum) {
            maxSum = sum;
            ans = arr[i];
        }
    }

    printf("\nThe integer with the highest digit sum is: %d", ans);

    return 0;
```

}

Status : Correct ***Marks : 10/10***

3. Problem Statement

Pavi is working on a project to help a library manage its collection of books efficiently. As part of this project, you need to develop a program to assist the library staff in finding specific books in their extensive catalog.

The library's book collection is organized in such a way that each book is assigned a unique numerical identifier. The library staff has provided you with a list of these identifiers, and you need to write a program to help them find a book's identifier efficiently.

Therefore, you decide to implement a recursive binary search algorithm to quickly locate books based on their unique identifiers.

Input Format

The first line of input contains an integer, n, representing the number of books in the library's collection.

The second line of input consists of n space-separated integers, where each integer represents the unique identifier of a book in the library's collection.

The third line of input contains a single integer t, representing the unique identifier of the book the library staff is searching for.

Output Format

If the book with the identifier target is found in the library's collection, the output prints the index (0-based) of the book in the collection.

If the book is not found, print "t not present", where the t is the unique identifier of a book.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 13 14 15 16
14

Output: 2

Answer

```
// You are using GCC
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int target) {
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (arr[mid] == target)
        return mid;

    if (arr[mid] > target)
        return binarySearch(arr, low, mid - 1, target);

    return binarySearch(arr, mid + 1, high, target);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int target;
    scanf("%d", &target);

    int result = binarySearch(arr, 0, n - 1, target);

    if (result == -1)
```

```
        printf("%d not present", target);
    else
        printf("%d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Jack is building a number analyzer tool that identifies numbers based on the sum of their digits. He is given a list of integers and wants to find the smallest number in the list whose digit sum equals a target value d.

To do this efficiently, Jack first sorts the list based on each number's digit sum and then applies binary search to find a match. Write a program to ensure that Jack can determine the correct number using this approach. If no number in the list matches the digit sum d, print -1.

Input Format

The first line of input consists of an integer n, representing the number of elements in the list.

The second line contains n space-separated integers, representing the sorted list.

The third line contains an integer d, representing the target digit sum.

Output Format

The output prints a single integer representing the smallest number from the list whose digit sum equals d, based on sorted digit sum order.

If no such number exists, print -1.

Refer to the sample output for formatting specifications.

vtu30672

Sample Test Case

Input: 5
12 23 30 48 56
3

Output: 12

Answer

```
// You are using GCC
#include <stdio.h>

int digitSum(int x) {
    int sum = 0;
    while (x > 0) {
        sum += x % 10;
        x /= 10;
    }
    return sum;
}

/* sort by digit sum, then value */
void sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            int ds1 = digitSum(a[j]);
            int ds2 = digitSum(a[j + 1]);

            if (ds1 > ds2 || (ds1 == ds2 && a[j] > a[j + 1])) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

/* binary search by digit sum */
int binarySearch(int a[], int n, int targetSum) {
    int low = 0, high = n - 1;
    int ans = -1;

    while (low <= high) {
```

```
        int mid = (low + high) / 2;
        int ds = digitSum(a[mid]);

        if (ds == targetSum) {
            ans = a[mid];
            high = mid - 1;  // search left for smallest number
        }
        else if (ds < targetSum) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    return ans;
}

int main() {
    int n;
    scanf("%d", &n);

    int a[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    int d;
    scanf("%d", &d);

    sort(a, n);

    int result = binarySearch(a, n, d);

    printf("%d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5
Output: 5th largest Salary: 28000

Answer

```
// You are using GCC
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
```

```
    *b = t;
}

void heapify(int heap[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && heap[left] > heap[largest])
        largest = left;

    if (right < n && heap[right] > heap[largest])
        largest = right;

    if (largest != i) {
        swap(&heap[i], &heap[largest]);
        heapify(heap, n, largest);
    }
}

void buildMaxHeap(int heap[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(heap, n, i);
    }
}

int extractMax(int heap[], int *n) {
    int max = heap[0];
    heap[0] = heap[*n - 1];
    (*n)--;
    heapify(heap, *n, 0);
    return max;
}

int findKthLargest(int arr[], int n, int k) {
    buildMaxHeap(arr, n);

    int size = n;
    int val = 0;

    for (int i = 0; i < k; i++) {
        val = extractMax(arr, &size);
```

```
    }
    return val;
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[102];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int k;
    scanf("%d", &k);

    int result = findKthLargest(arr, n, k);

    printf("%dth largest Salary: %d", k, result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 5_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48

Section 1 : Coding

1. Problem Statement

Suppose you are working on a project that involves sorting a large dataset of student records based on a single criterion, such as their CGPA, age, or major. You have decided to use a modified quicksort algorithm to efficiently sort the dataset while ensuring that the sorting is stable, i.e., that records with equal values maintain their original relative order.

To achieve this, you will implement a modified version of the quicksort algorithm that maintains stability during the sorting process. Specifically, you will use a three-way partitioning scheme that sorts the records into three groups: those with values less than the pivot, those with values equal to the pivot, and those with values greater than the pivot. When the algorithm processes subarrays with equal pivot values, it uses insertion sort to maintain stability.

vtu30672

Write a function that implements this modified version of the quicksort algorithm and uses it to sort the student records according to one specified criterion (CGPA, age, or major). Your function should take as input a list of student records and return the sorted list in ascending order based on the chosen criterion. If two records have the same values for the sorting criterion, their relative order should be maintained in the sorted list.

Input Format

The first line contains an integer n, representing the number of student records to be sorted.

The next n groups of 4 lines each contain the details of each student:

- Line 1: Name (string)
- Line 2: CGPA (float)
- Line 3: Age (integer)
- Line 4: Major (string)

The last line contains the sorting criteria (one of three options: "CGPA", "age", or "major").

Output Format

The output displays the sorted student records in a table format with the following specifications:

Header row: "Name CGPA Age Major"

Each student record is displayed in one row with the following format:

- Name: Left-aligned, 10 characters wide
- CGPA: Left-aligned, 8 characters wide, displayed with 2 decimal places
- Age: Left-aligned, 5 characters wide
- Major: Left-aligned, 20 characters wide

The table is sorted in ascending order according to the chosen sorting criteria.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 4

vtu30672

harish
7.8
23
ece
trello
8
21
cse
kamalesh
7
25
it
maaran
5.6
20
civil
age

Output: Name CGPA Age Major
maaran 5.60 20 civil
trello 8.00 21 cse
harish 7.80 23 ece
kamalesh 7.00 25 it

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct {
    char name[51];
    float cgpa;
    int age;
    char major[51];
} Student;

char criterion[20];

int compare(Student a, Student b) {
    if (strcmp(criterion, "age") == 0)
        return a.age - b.age;
```

```
    if (strcmp(criterion, "cgpa") == 0 || strcmp(criterion, "gpa") == 0) {
        if (a.cgpa < b.cgpa) return -1;
        if (a.cgpa > b.cgpa) return 1;
        return 0;
    }

    // major
    return strcmp(a.major, b.major);
}

/* Stable 3-way quicksort */
void quicksort(Student arr[], int n) {
    if (n <= 1) return;

    Student pivot = arr[0];

    Student *less = (Student*)malloc(n * sizeof(Student));
    Student *equal = (Student*)malloc(n * sizeof(Student));
    Student *greater = (Student*)malloc(n * sizeof(Student));

    int l = 0, e = 0, g = 0;

    for (int i = 0; i < n; i++) {
        int cmp = compare(arr[i], pivot);

        if (cmp < 0)
            less[l++] = arr[i];
        else if (cmp > 0)
            greater[g++] = arr[i];
        else
            equal[e++] = arr[i];
    }

    quicksort(less, l);
    quicksort(greater, g);

    int idx = 0;

    for (int i = 0; i < l; i++) arr[idx++] = less[i];
    for (int i = 0; i < e; i++) arr[idx++] = equal[i];
    for (int i = 0; i < g; i++) arr[idx++] = greater[i];
```

```
    free(less);
    free(equal);
    free(greater);
}

int main() {
    int n;
    scanf("%d", &n);

    Student *arr = (Student*)malloc(n * sizeof(Student));

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i].name);
        scanf("%f", &arr[i].cgpa);
        scanf("%d", &arr[i].age);
        scanf("%s", arr[i].major);
    }

    scanf("%s", criterion);

    for (int i = 0; criterion[i]; i++)
        criterion[i] = tolower(criterion[i]);

    quicksort(arr, n);

    printf("Name      CGPA     Age   Major            \n");

    for (int i = 0; i < n; i++) {
        printf("%-10s%-8.2f%-5d%-20s\n",
               arr[i].name,
               arr[i].cgpa,
               arr[i].age,
               arr[i].major);
    }

    free(arr);
    return 0;
}
```

Status : Partially correct ***Marks : 8/10***

2. Problem Statement

Vishnu, a math enthusiast, is given a task to explore the magic of numbers. He has an array of positive integers, and his goal is to find the integer with the highest digit sum in the sorted array using the merge sort algorithm.

You have to assist Vishnu in implementing the merge sort algorithm.

Example

Input:

6

789 123 456 876 234 567

Output:

The sorted array is: 123 234 456 567 789 876

The integer with the highest digit sum is: 789

Explanation

Sorted Array Output: 123 234 456 567 789 876

The digit sum of an integer is the sum of its individual digits.

In this case, the integer 789 has the highest digit sum (7 + 8 + 9 = 24).

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the array elements.

Output Format

The first line prints "The sorted array is: " followed by the sorted array in ascending order.

The second line prints "The integer with the highest digit sum is: " followed by the integer with the highest-digit sum.

vtu30672

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
123 456 789 321 654

Output: The sorted array is: 123 321 456 654 789
The integer with the highest digit sum is: 789

Answer

```
// You are using GCC
#include <stdio.h>

int arr[10], temp[10];
int n;

int digitSum(int x) {
    int sum = 0;
    while (x > 0) {
        sum += x % 10;
        x /= 10;
    }
    return sum;
}

void merge(int low, int mid, int high) {
    int i = low, j = mid + 1, k = low;

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= high)
```

```
        temp[k++] = arr[j++];

    for (i = low; i <= high; i++)
        arr[i] = temp[i];
}

void mergeSort(int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(low, mid);
        mergeSort(mid + 1, high);
        merge(low, mid, high);
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    mergeSort(0, n - 1);

    printf("The sorted array is: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    int maxSum = -1, ans = arr[0];

    for (int i = 0; i < n; i++) {
        int sum = digitSum(arr[i]);
        if (sum > maxSum) {
            maxSum = sum;
            ans = arr[i];
        }
    }

    printf("\nThe integer with the highest digit sum is: %d", ans);

    return 0;
```

}

Status : Correct ***Marks : 10/10***

3. Problem Statement

Pavi is working on a project to help a library manage its collection of books efficiently. As part of this project, you need to develop a program to assist the library staff in finding specific books in their extensive catalog.

The library's book collection is organized in such a way that each book is assigned a unique numerical identifier. The library staff has provided you with a list of these identifiers, and you need to write a program to help them find a book's identifier efficiently.

Therefore, you decide to implement a recursive binary search algorithm to quickly locate books based on their unique identifiers.

Input Format

The first line of input contains an integer, n, representing the number of books in the library's collection.

The second line of input consists of n space-separated integers, where each integer represents the unique identifier of a book in the library's collection.

The third line of input contains a single integer t, representing the unique identifier of the book the library staff is searching for.

Output Format

If the book with the identifier target is found in the library's collection, the output prints the index (0-based) of the book in the collection.

If the book is not found, print "t not present", where the t is the unique identifier of a book.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 13 14 15 16
14

Output: 2

Answer

```
// You are using GCC
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int target) {
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (arr[mid] == target)
        return mid;

    if (arr[mid] > target)
        return binarySearch(arr, low, mid - 1, target);

    return binarySearch(arr, mid + 1, high, target);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int target;
    scanf("%d", &target);

    int result = binarySearch(arr, 0, n - 1, target);

    if (result == -1)
```

```
        printf("%d not present", target);
    else
        printf("%d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Jack is building a number analyzer tool that identifies numbers based on the sum of their digits. He is given a list of integers and wants to find the smallest number in the list whose digit sum equals a target value d.

To do this efficiently, Jack first sorts the list based on each number's digit sum and then applies binary search to find a match. Write a program to ensure that Jack can determine the correct number using this approach. If no number in the list matches the digit sum d, print -1.

Input Format

The first line of input consists of an integer n, representing the number of elements in the list.

The second line contains n space-separated integers, representing the sorted list.

The third line contains an integer d, representing the target digit sum.

Output Format

The output prints a single integer representing the smallest number from the list whose digit sum equals d, based on sorted digit sum order.

If no such number exists, print -1.

Refer to the sample output for formatting specifications.

vtu30672

Sample Test Case

Input: 5
12 23 30 48 56
3

Output: 12

Answer

```
// You are using GCC
#include <stdio.h>

int digitSum(int x) {
    int sum = 0;
    while (x > 0) {
        sum += x % 10;
        x /= 10;
    }
    return sum;
}

/* sort by digit sum, then value */
void sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            int ds1 = digitSum(a[j]);
            int ds2 = digitSum(a[j + 1]);

            if (ds1 > ds2 || (ds1 == ds2 && a[j] > a[j + 1])) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

/* binary search by digit sum */
int binarySearch(int a[], int n, int targetSum) {
    int low = 0, high = n - 1;
    int ans = -1;

    while (low <= high) {
```

```
        int mid = (low + high) / 2;
        int ds = digitSum(a[mid]);

        if (ds == targetSum) {
            ans = a[mid];
            high = mid - 1;  // search left for smallest number
        }
        else if (ds < targetSum) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    return ans;
}

int main() {
    int n;
    scanf("%d", &n);

    int a[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    int d;
    scanf("%d", &d);

    sort(a, n);

    int result = binarySearch(a, n, d);

    printf("%d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5
Output: 5th largest Salary: 28000

Answer

```
// You are using GCC
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
```

```
    *b = t;
}

void heapify(int heap[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && heap[left] > heap[largest])
        largest = left;

    if (right < n && heap[right] > heap[largest])
        largest = right;

    if (largest != i) {
        swap(&heap[i], &heap[largest]);
        heapify(heap, n, largest);
    }
}

void buildMaxHeap(int heap[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(heap, n, i);
    }
}

int extractMax(int heap[], int *n) {
    int max = heap[0];
    heap[0] = heap[*n - 1];
    (*n)--;
    heapify(heap, *n, 0);
    return max;
}

int findKthLargest(int arr[], int n, int k) {
    buildMaxHeap(arr, n);

    int size = n;
    int val = 0;

    for (int i = 0; i < k; i++) {
        val = extractMax(arr, &size);
```

```
    }
    return val;
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[102];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int k;
    scanf("%d", &k);

    int result = findKthLargest(arr, n, k);

    printf("%dth largest Salary: %d", k, result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 6_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Ankit wants to share the job among his employees so that maximum profit can be obtained. The condition is that no two jobs for a particular employee overlap.

Given N jobs, where every job is represented by the following three elements:

Start Time

Finish Time

Profit or Value Associated (>= 0)

Find the maximum-profit subset of jobs such that no two jobs in the subset overlap.

vtu30672

Input Format

The first line of the input consists of the value of n.

The next n lines of the inputs consist of the start, end, and profit.

Output Format

The output prints the optimal profit.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: 4
3 10 20
1 2 50
6 19 100
2 100 200
Output: The optimal profit is 250

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Structure to store job details
struct Job {
    int start, finish, profit;
};

// Comparator function to sort jobs by finish time
int compare(const void *a, const void *b) {
    struct Job *job1 = (struct Job *)a;
    struct Job *job2 = (struct Job *)b;
    return job1->finish - job2->finish;
}

// Binary search to find the last non-conflicting job
int latestNonConflict(struct Job jobs[], int i) {
    int low = 0, high = i - 1;
```

```
    while (low <= high) {
        int mid = (low + high) / 2;

        if (jobs[mid].finish <= jobs[i].start) {
            if (jobs[mid + 1].finish <= jobs[i].start)
                low = mid + 1;
            else
                return mid;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

// Function to calculate maximum profit
int findMaxProfit(struct Job jobs[], int n) {
    // Sort jobs by finish time
    qsort(jobs, n, sizeof(struct Job), compare);

    int dp[n];
    dp[0] = jobs[0].profit;

    for (int i = 1; i < n; i++) {
        int inclProfit = jobs[i].profit;

        int l = latestNonConflict(jobs, i);
        if (l != -1)
            inclProfit += dp[l];

        dp[i] = (inclProfit > dp[i - 1]) ? inclProfit : dp[i - 1];
    }

    return dp[n - 1];
}

int main() {
    int n;
    scanf("%d", &n);

    struct Job jobs[n];
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &jobs[i].start, &jobs[i].finish, &jobs[i].profit);
    }

    int result = findMaxProfit(jobs, n);

    printf("The optimal profit is %d\n", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

In a city, there are several locations connected by roads. Each road has a certain distance. You are tasked with finding the shortest distances between all pairs of locations in the city using the Floyd-Warshall algorithm.

Write a program that calculates the shortest path between all pairs of locations in the city.

Example

Input:

4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output:

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Explanation:

The city has 4 locations (0 to 3).

There are 4 roads connecting the locations.

The roads and their distances are:

0 1 with distance 3

1 2 with distance 1

2 3 with distance 1

0 2 with distance 5

Initially, the distance matrix is set to infinity (INF) except for direct roads and diagonal values (0).

The Floyd-Warshall algorithm updates the matrix by checking intermediate paths.

The shortest path from 0 to 2 (via 1) is updated to 4 (0 1 2) instead of 5.

The shortest path from 0 to 3 is 5 (0 1 2 3).

The final distance matrix is printed, showing the shortest distances.

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Input Format

The first line contains an integer N, representing the number of locations in the city.

The second line contains an integer M, representing the number of roads in the city.

The next M lines each contain three space-separated integers: u, v, w where:

- u and v represent two locations connected by a road.
- w represents the distance between u and v.

Output Format

The output prints a matrix, where the value at the i-th row and j-th column represents the shortest distance between location i and location j.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
4
0 1 3
1 2 1
2 3 1
0 2 5

Output: 0 3 4 5
3 0 1 2
4 1 0 1
5 2 1 0

Answer

```
#include <stdio.h>

#define INF 100000

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    int dist[n][n];

    // Initialize distance matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                dist[i][j] = 0;
```

```
        else
            dist[i][j] = INF;
    }
}

// Read edges
for (int i = 0; i < m; i++) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);
    dist[u][v] = w;
    dist[v][u] = w; // undirected graph
}

// Floyd-Warshall Algorithm
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

// Print result
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", dist[i][j]);
    }
    printf("\n");
}

return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Sharon is developing a program that analyzes the reachability of nodes in a graph. Given an adjacency matrix representing the graph and its size, her

goal is to determine whether there exists a path from one vertex to another for all pairs of vertices.

Write a program to help Sharon perform this reachability analysis using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines consist of N space-separated integers, forming the adjacency matrix graph, where graph[i][j] is either 0 or 1.

Output Format

The output prints an NxN matrix representing the reachability matrix for all pairs of vertices.

Each element in the matrix should be 1 if there is a path from the corresponding row vertex to the column vertex, and 0 otherwise.

Sample Test Case

Input: 4
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0
Output: 0 1 1 1
0 0 1 1
0 0 0 1
0 0 0 0

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int graph[n][n];
```

```
    // Input adjacency matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // Warshall's Algorithm
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
            }
        }
    }

    // Output reachability matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", graph[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

You and your group of adventurous friends are planning a once-in-a-lifetime outdoor adventure trip to an exotic location. To ensure that your adventure goes smoothly, you need to carefully choose what to carry with you, as you have a limited carrying capacity. Your goal is to maximize the overall value of the items you bring while staying within your weight limit.

Write a program that helps you and your friends determine the optimal selection of items to bring on your adventure trip. The program should implement the 0–1 Knapsack Problem algorithm to find the best

combination of items to maximize their total value while respecting the weight constraint.

Input Format

The first line contains an integer n, representing the number of items available for selection.

The second line contains an integer limit, representing the maximum weight limit your group can carry in a bag.

The next n lines contain space-separated information for each item:

- The name of the item is a string (without spaces).
- The weight of the item.
- The value of the item.

Output Format

The first line of output displays "Total value: " followed by an integer representing the total value of the selected items.

The second line of output displays "Total weight: " followed by an integer representing the total weight of the selected items.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 2
25
Bag 10 250
Note 15 430

Output: Total value: 680
Total weight: 25

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
int main() {
    int n, limit;
    scanf("%d", &n);
    scanf("%d", &limit);

    char name[n][50];
    int weight[n], value[n];

    // Input items
    for (int i = 0; i < n; i++) {
        scanf("%s %d %d", name[i], &weight[i], &value[i]);
    }

    // DP table
    int dp[n + 1][limit + 1];

    // Initialize DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= limit; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (weight[i - 1] <= w) {
                int include = value[i - 1] + dp[i - 1][w - weight[i - 1]];
                int exclude = dp[i - 1][w];
                dp[i][w] = (include > exclude) ? include : exclude;
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    // Find total weight of selected items
    int res = dp[n][limit];
    int w = limit;
    int totalWeight = 0;

    for (int i = n; i > 0 && res > 0; i--) {
        if (res != dp[i - 1][w]) {
            totalWeight += weight[i - 1];
            res -= value[i - 1];
            w -= weight[i - 1];
        }
```

```
    }

    printf("Total value: %d\n", dp[n][limit]);
    printf("Total weight: %d\n", totalWeight);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 6_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 44.5

Section 1 : Coding

1. Problem Statement

Virat is given a group of individuals, and he wants to identify if there is an influential individual within the group. An influential individual is defined as someone who is followed by more people than they follow.

He is provided with a list of follow-up relationships among the individuals in the form of a matrix. Your task is to help Virat determine if there is an influential individual and, if so, find their index within the group.

Write a program to solve this problem using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of individuals in the group.

vtu30672

The following N lines will contain N space-separated integers, representing the following relationship matrix for the individuals.

Output Format

If an influential individual is found, print "Influential Individual: X," where X is the index (0-based) of the influential individual.

Otherwise, print "There is no influential individual in the group".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 1 1 0
0 0 1 0
0 0 0 0
0 0 1 0
Output: Influential Individual: 2

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int graph[n][n], original[n][n];

    // Input
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            original[i][j] = graph[i][j];
        }
    }

    // Warshall's Algorithm (not used for counting, only requirement)
    for (int k = 0; k < n; k++) {
```

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
        }
    }
}

int found = 0;

for (int i = 0; i < n; i++) {
    int followers = 0, following = 0;

    for (int j = 0; j < n; j++) {
        if (i != j) {
            if (original[j][i] == 1)
                followers++;
            if (original[i][j] == 1)
                following++;
        }
    }

    //    FIX: must have at least one follower
    if (followers > following && followers > 0) {
        printf("Influential Individual: %d", i);
        found = 1;
        break;
    }
}

if (!found) {
    printf("There is no influential individual in the group");
}

return 0;
}
```

Status : Partially correct ***Marks : 4.5/10***

2. Problem statement

Sarah has been tasked with developing a program to determine the

shortest paths among nodes in a directed network represented as a graph containing nodes and weighted edges.

Her goal is to implement the Floyd-Warshall algorithm to compute the shortest path matrix for this directed network.

Input Format

The first line of input consists of an integer n, representing the number of nodes in the network.

The next n lines contain a space-separated integer, representing the adjacency matrix 'graph' of the network. Each integer is the weight of the edge from node i to node j.

Output Format

The output prints the shortest path matrix as 'n x n' integers, where each element graph[i][j] represents the shortest distance from node 'i' to node 'j'.

Print each row of the matrix on a new line, with elements separated by a space, and if no path exists, 999 will be displayed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 3 999 4
8 0 2 999
5 999 0 1
2 999 999 0

Output: 0 3 5 4
5 0 2 3
3 6 0 1
2 5 7 0

Answer

```
// You are using GCC
#include <stdio.h>
```

vtu30672

```
#define INF 999

int main() {
    int n;
    scanf("%d", &n);

    int dist[n][n];

    // Input adjacency matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }

    // Floyd-Warshall Algorithm
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                // Avoid overflow and invalid paths
                if (dist[i][k] != INF && dist[k][j] != INF) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }
    }

    // Output result
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

You are working as a backend engineer for a drone delivery logistics company that operates across multiple cities. Each city is represented as a node, and the drone flight paths between cities are represented as directed edges with weights indicating the time (in minutes) it takes for a drone to travel between them.

Due to varying air traffic, regulations, and geography, the direct travel time between cities may not always be the fastest. Sometimes, going through intermediate cities results in shorter delivery times.

Your task is to build a system that calculates the shortest possible delivery time between every pair of cities, considering all possible intermediate routes.

To efficiently compute this, you're required to implement the Floyd-Warshall Algorithm.

Input Format

The first line of input contains an integer N representing the total number of cities.

The next N lines contains N space-separated integers, representing the delivery time matrix:

time[i][j] is the time (in minutes) taken to fly directly from City i to City j.

A value of 0 means the same city (i.e., i == j).

A value of 999 indicates no direct route between City i and City j.

Output Format

The output prints the shortest distance between every pair of cities.

If there is no shortest path between two cities, then print INF.

vtu30672

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

Output: 0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Answer

```
// You are using GCC
#include <stdio.h>

#define INF 999999

int main() {
    int n;
    scanf("%d", &n);

    int dist[n][n];

    // Input matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }

    // Floyd-Warshall Algorithm
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {

                if (dist[i][k] != 999 && dist[k][j] != 999) {
                    if (dist[i][k] + dist[k][j] < dist[i][j] || dist[i][j] == 999) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
```

```
            }
        }
    }

    // Output result
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {

            if (dist[i][j] == 999) {
                printf("INF ");
            } else {
                printf("%d ", dist[i][j]);
            }

        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays, val[0..n-1] and wt[0..n-1] which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of val[] such that the sum of the weights of this subset is smaller than or equal to W. You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

Input Format

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

Output Format

The output prints the maximum value of items that can be held by the knapsack.

Sample Test Case

Input: 3
60 100 120
10 20 30
50
Output: 220

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int val[n], wt[n];

    // Input values
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }

    // Input weights
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }
```

```
    int W;
    scanf("%d", &W);

    int dp[n + 1][W + 1];

    // DP initialization
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w) {
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    // Output result
    printf("%d\n", dp[n][W]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Sarah loves challenges and has recently taken up a new programming problem. She is fascinated by the classic "0/1 Knapsack Problem" and wants to create a program that can efficiently solve it.

The 0/1 Knapsack Problem involves selecting a combination of items with given weights and values to maximize the total value, while not exceeding a given weight capacity.

Write a program that takes input for the 0/1 Knapsack Problem and outputs the maximum value that can be achieved, as well as the selected weights that contribute to this maximum value.

Note: The weights are printed in reverse order of the input, meaning the algorithm starts checking from the last item and prints items that are included first.

Input Format

The first line contains an integer n, representing the number of items available.

The second line contains n integers separated by space, representing the values of the items. The ith integer corresponds to the value of the ith item.

The third line contains n integers separated by space, representing the weights of the items. The ith integer corresponds to the weight of the ith item.

The fourth line contains an integer W, representing the maximum weight capacity of the knapsack.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, representing the selected weights contributing to the maximum value.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: Maximum value: 220
Selected weights: 30 20

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int val[n], wt[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }

    int W;
    scanf("%d", &W);

    int dp[n + 1][W + 1];

    // Build DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    printf("Maximum value: %d\n", dp[n][W]);

    // Backtracking to find selected weights (reverse order)
    int w = W;
    printf("Selected weights: ");
```

```
    for (int i = n; i > 0 && w > 0; i--) {
        if (dp[i][w] != dp[i - 1][w]) {
            printf("%d ", wt[i - 1]);
            w -= wt[i - 1];
        }
    }

    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MANDLA VENKATA MADHAN Madhan
Email: vtu30672@veltech.edu.in
Roll no: vtu30672
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS4L11

VTU_DAA_Week 6_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Anitha is working on packing expensive artifacts into a container that has a limited weight capacity. Each artifact has a weight and a value, and Anitha wants to maximize the total worth of artifacts she can carry, even if it means taking only a fraction of an artifact when the container is nearly full.

The maximum worth is calculated as:

Maximum Worth = Σ (value[i] * fraction_taken[i]),

where fraction_taken[i] = min(1, remaining_capacity / weight[i])

Write a program to ensure that Anitha can calculate the maximum worth of objects she can carry in the container.

vtu30672

Input Format

The first line contains an integer N, representing the number of items.

The second line contains N space-separated integers representing the weights of the items.

The third line contains N space-separated integers representing the values of the items.

The fourth line contains an integer representing the capacity of the container.

Output Format

The output prints a single line showing the total maximum worth of objects carried as a double with exactly two decimal places, in the following format:

"Objects worth Rs.X.YY"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
60 100 120
50
Output: Objects worth Rs.240.00

Answer

```
// You are using GCC
#include <stdio.h>

typedef struct {
   int weight;
   int value;
   double ratio;
} Item;
```

vtu30672

```
// Swap function
void swap(Item *a, Item *b) {
    Item temp = *a;
    *a = *b;
    *b = temp;
}

// Sort by value/weight ratio (descending)
void sort(Item arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i].ratio < arr[j].ratio) {
                swap(&arr[i], &arr[j]);
            }
        }
    }
}

int main() {
    int n;
    scanf("%d", &n);

    Item items[n];

    // Input weights
    for (int i = 0; i < n; i++) {
        scanf("%d", &items[i].weight);
    }

    // Input values
    for (int i = 0; i < n; i++) {
        scanf("%d", &items[i].value);
        items[i].ratio = (double)items[i].value / items[i].weight;
    }

    int capacity;
    scanf("%d", &capacity);

    // Sort by ratio
    sort(items, n);
```

```
    double totalValue = 0.0;

    // Greedy selection
    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {
            capacity -= items[i].weight;
            totalValue += items[i].value;
        } else {
            totalValue += items[i].ratio * capacity;
            break;
        }
    }

    printf("Objects worth Rs.%.2f\n", totalValue);

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Raju is a professional hiker preparing for a long trekking expedition. He has a backpack that can hold a maximum weight of W kilograms. There are N essential items you want to carry, each with a specific weight and importance value (utility score). His goal is to maximize the total importance value of the items you take while ensuring that the total weight does not exceed the backpack's capacity.

He can either include or exclude each item—meaning he cannot take fractions of an item (e.g., you cannot take half of a tent). Given the weights and importance values of the items, determine the maximum possible importance value he can carry in his backpack. Help him to achieve his task.

Input Format

The first line contains two integers, N (number of items) and W (maximum weight the backpack can carry).

The second line contains N integers, representing the weights of the items.

The third line contains N integers, representing the importance values of the items.

Output Format

The output prints a single integer, the maximum importance value that can be achieved without exceeding the weight limit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 5
2 3 4 5
3 4 5 6

Output: 7

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    scanf("%d %d", &n, &W);

    int wt[n], val[n];

    // Input weights
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }

    // Input values
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
```

```
    }
    int dp[n + 1][W + 1];

    // Build DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;

            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);

            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    printf("%d\n", dp[n][W]);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Rohit is responsible for scheduling flights for an airline company. He has a list of N flights numbered from 0 to N-1. Some flights have specific constraints that require them to depart after other flights.

Write a program to help Rohit determine if it's possible to create a flight schedule that adheres to these constraints using Warshall's algorithm.

Example 1

Input:

5

0 1

1 2

2 3

3 4

4 5

Output:

Yes

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

There are no circular dependencies or conflicts in the schedule, so it is possible to create a flight schedule that adheres to the constraints.

Example 2

Input:

6

0 1

1 2

2 3

3 4

4 5

5 0

Output:

No

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

Flight 5 must depart after Flight 0.

There is a circular dependency in the flight constraints. Therefore, it is not possible to create a flight schedule that follows the constraints.

Input Format

The first line of input consists of an integer N, representing the number of flights.

The following N lines consist of two space-separated integers each: the flight number and the flight number that must depart before it.

Output Format

If it is possible to create a flight schedule that adheres to the constraints, print "Yes".

Otherwise, print "No".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 1
1 2
2 3
3 4
4 5

vtu30672

Output: Yes

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int graph[n][n];

    // Initialize matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = 0;
        }
    }

    // Read edges (u must come before v => edge u -> v)
    for (int i = 0; i < n; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    // Warshall's Algorithm (transitive closure)
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
            }
        }
    }

    // Check for cycle (self reachability)
    for (int i = 0; i < n; i++) {
        if (graph[i][i] == 1) {
            printf("No");
            return 0;
        }
    }
```

```
    printf("Yes");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

A traveler needs to find the shortest path between every pair of cities in a road network. The cities are connected by roads, each with a specific travel cost. To do this, the traveler uses the Floyd-Warshall Algorithm to calculate the shortest distances between every pair of cities in a given edge-weighted undirected graph.

Example1

Input:

4

3

0 1 2

1 2 3

2 3 4

Output:

Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Explanation:

Given: 4 nodes (0, 1, 2, 3) and 3 edges: 0 1 (weight 2), 1 2 (weight 3), 2 3 (weight 4)

Any node pair without a direct edge is considered INF (unreachable initially).

Step 1: Construct Initial Graph (Original Matrix)

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Diagonal elements (self to self) are 0.If no direct path exists, it's INF.Step 2: Running Floyd-Warshall Algorithm

Using node 0 as an intermediate: No updates since it only connects to node 1.

Using node 1 as an intermediate: 0 2 via 1: 0 1 (2) + 1 2 (3) = 5

2 0 also updates to 5.

Using node 2 as an intermediate: 0 3 via 2: 0 2 (5) + 2 3 (4) = 9

1 3 via 2: 1 2 (3) + 2 3 (4) = 7

Using node 3 as an intermediate: No further updates.

Step 3: Final Shortest Path Matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

vtu30672

Input Format

The first line contains an integer V representing the number of cities (vertices) in the network.

The second line contains an integer E representing the number of roads (edges) in the network.

The next E lines each contain three space-separated integers u, v, w, where u and v represent the cities connected by a road, and w represents the travel cost (weight) of the road.

Output Format

The first line of output prints the original matrix, where each entry represents the direct travel cost between two cities. If there is no direct road between two cities, represent the cost as INF.

Then, print the shortest path matrix, where each entry represents the shortest travel cost between every pair of cities, computed using the Floyd-Warshall algorithm.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
3
0 1 2
1 2 3
2 3 4

Output: Original matrix
0 2 INF INF
2 0 3 INF
INF 3 0 4
INF INF 4 0

Shortest path matrix
0 2 5 9
2 0 3 7
5 3 0 4

vtu30672

9 7 4 0

Answer

```
// You are using GCC
#include <stdio.h>

#define INF 1000000000

int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int V, E;
    scanf("%d", &V);
    scanf("%d", &E);

    int dist[V][V];

    // Initialize matrix
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (i == j)
                dist[i][j] = 0;
            else
                dist[i][j] = INF;
        }
    }

    // Input edges (undirected graph)
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);

        if (w < dist[u][v]) {
            dist[u][v] = w;
            dist[v][u] = w;
        }
    }

    // Print Original Matrix
    printf("Original matrix\n");
```

```
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    // Floyd-Warshall Algorithm
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INF && dist[k][j] != INF) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }

    // Print Shortest Path Matrix
    printf("\nShortest path matrix\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

A person starts walking from position X = 0, find the probability of reaching exactly on X = N if she can only take either 2 steps or 3 steps. The probability for step length 2 is given i.e. P, and the probability for step length 3 is 1 – P.

Write a program for the same.

Note: Use Dynamic Programming

Example

Input: N = 5, P = 0.20

Output: 0.32

Explanation:

There are two ways to reach 5.

2+3 with probability = 0.2 * 0.8 = 0.16

3+2 with probability = 0.8 * 0.2 = 0.16

So, total probability = 0.32.

Input Format

The input consists of an integer N representing the target position and a double-point number P representing the probability of taking a step of length 2, separated by a space.

Output Format

The output prints a double-point number representing the probability of reaching the exact position N, formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 0.20
Output: 0.32

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
```

```
    int N;
    double P;

    scanf("%d %lf", &N, &P);

    double dp[N + 1];

    // Initialize
    for (int i = 0; i <= N; i++)
        dp[i] = 0.0;

    dp[0] = 1.0;

    // DP computation
    for (int i = 1; i <= N; i++) {
        if (i - 2 >= 0)
            dp[i] += dp[i - 2] * P;

        if (i - 3 >= 0)
            dp[i] += dp[i - 3] * (1.0 - P);
    }

    printf("%.2f\n", dp[N]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***